VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



VO ANH KIET - 523H0048

GRADUATION THESIS

DEVELOPMENT OF A REAL-TIME COLLABORATIVE WORKSPACE PLATFORM WITH INTEGRATED MULTIPOINT VIDEO CONFERENCING

HO CHI MINH CITY, YEAR 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



VO ANH KIET - 523H0048

GRADUATION THESIS

DEVELOPMENT OF A REAL-TIME COLLABORATIVE WORKSPACE PLATFORM WITH INTEGRATED MULTIPOINT VIDEO CONFERENCING

Advised by
Vu Dinh Hong

HO CHI MINH CITY, YEAR 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Mr. Vu Dinh Hong, my instructor and mentor, for his guidance throughout this graduation thesis. His valuable suggestions, his patience and his constructive feedback on the design, the experiments and this report greatly contributed to the completion of the project.

I also thank the lecturers of the Faculty of Information Technology, Ton Duc Thang University, for the knowledge they shared during my studies, and my family and friends, who supported me and helped test the application. Despite my efforts, the thesis certainly still has shortcomings, and I look forward to receiving comments from the lecturers.

Ho Chi Minh City, October 2026

Author

(Signature and full name)

Vo Anh Kiet

DECLARATION OF AUTHORSHIP

I hereby declare that this graduation thesis is my own work, carried out under the guidance of Mr. Vu Dinh Hong. The research content and the results presented in it are original and have not been published in any form before. The measurements in the tables and figures used for analysis and evaluation were collected by me from the experiments described in Chapter 5, and the sources are stated clearly in the report.

The thesis also uses ideas, specifications and data from other authors and organisations; all of them are cited and listed in the references.

If any fraud is found, I take full responsibility for the content of my thesis. Ton Duc Thang University is not involved in any copyright violation that I may have caused in the course of this work.

Ho Chi Minh City, October 2026

Author

(Signature and full name)

Vo Anh Kiet

ABSTRACT

Remote teams, online classes and online communities usually split their work across several tools: a chat application for day-to-day discussion, a separate service for video meetings and another one for task tracking. Every switch between these tools breaks the context of the conversation. This thesis designs, implements and evaluates **nexo**, a browser-based collaborative workspace that combines channel-based real-time messaging, presence, multipoint video conferencing and lightweight task management in a single application organised as *Workspace → Channel*.

The system consists of a React/TypeScript single-page client, a Node.js application server that carries REST, Socket.IO messaging and WebRTC signalling, a mediasoup Selective Forwarding Unit (SFU) with a pool of C++ media workers, and a PostgreSQL database that also synchronises events and presence between application processes. Messages are delivered with an idempotent protocol (client-generated message identifiers, a durable client outbox and a database uniqueness constraint), so that lost acknowledgements and reconnections neither lose nor duplicate messages. The conferencing client uses three-layer VP8 simulcast and a dynamic video grid that only requests the layers it can actually display.

The system was evaluated with automated tests, a WebSocket load test and a media laboratory that runs every participant as a separate headless Chromium instance. With 200 concurrent sockets the 95th-percentile message delivery latency stayed below 50 ms with no lost or duplicated messages. Against a full-mesh peer-to-peer implementation used as the baseline on the same hardware, the SFU kept the per-client uplink almost constant as the room grew, while the mesh client's uplink and CPU grew linearly and the mesh session collapsed at six participants. All experiments ran on a single machine over the loopback network; multi-network, TURN and production-scale measurements remain future work.

Keywords: real-time collaboration, WebSocket, Socket.IO, WebRTC, SFU, mediasoup, simulcast, QoS.

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION AND OVERVIEW 1

1.1 Reasons for choosing the topic 1

1.2 Problem statement 1

1.3 Objectives 2

1.4 Scope 2

1.4.1 Target users 2

1.4.2 In scope 3

1.4.3 Out of scope 3

1.5 Methodology 3

1.6 Technologies used 4

1.7 Report structure 4

CHAPTER 2. THEORETICAL BACKGROUND 6

2.1 Real-time web communication 6

2.1.1 From request–response to WebSocket 6

2.1.2 Socket.IO 7

2.1.3 Presence synchronisation 7

2.2 WebRTC 8

2.2.1 Session negotiation with SDP 8

2.2.2 NAT traversal: ICE, STUN and TURN 8

2.2.3 Security: DTLS-SRTP 9

2.2.4 RTP, RTCP and QoS metrics 10

2.2.5 Congestion control, simulcast and codecs 10

2.3 Multipoint conferencing architectures 11

2.3.1 Mesh *11*

2.3.2 MCU *11*

2.3.3 SFU *12*

2.3.4 Quantitative comparison *12*

2.4 The mediasoup SFU 13

2.5 Role-based access control 13

2.6 Chapter summary 14

CHAPTER 3. SYSTEM BLOCK DIAGRAM AND DESIGN 15

3.1 Requirements analysis 15

3.1.1 Actors *15*

3.1.2 Functional requirements *15*

3.1.3 Non-functional requirements *16*

3.1.4 Main use cases *17*

3.2 System block diagram 17

3.2.1 Overview *17*

3.2.2 Control plane and data plane *19*

3.3 Block analysis 19

3.3.1 Client block *19*

3.3.2 Application and signalling server block *20*

3.3.3 SFU media block *20*

3.3.4 Data block *21*

3.3.5 Network traversal block *21*

3.4 Database design....21

3.5 Access control design....23

3.6 Real-time protocol design....24

3.6.1 Event catalogue....24

3.6.2 Message delivery flow....24

3.6.3 Joining a video channel....25

3.7 Horizontal scaling design....26

3.8 User interface design....27

3.9 Chapter summary....27

CHAPTER 4. SYSTEM IMPLEMENTATION....28

4.1 Source code organisation....28

4.2 Real-time core....29

4.2.1 Authentication and per-event authorisation....29

4.2.2 Exactly-once message storage....30

4.2.3 Presence and typing....32

4.2.4 Multi-process event distribution....33

4.3 Conferencing core....34

4.3.1 Worker pool and room placement....34

4.3.2 Producer and consumer lifecycle....34

4.3.3 Meeting sessions and moderation....35

4.3.4 NAT traversal and ICE recovery....35

4.4 Conferencing client and dynamic video grid....36

4.4.1 Capture and publishing....36

4.4.2 QoS collection *37*

4.4.3 Dynamic grid and demand-driven layers *37*

4.5 User interface 39

4.6 Deployment 43

4.7 Chapter summary 44

CHAPTER 5. EXPERIMENTS AND EVALUATION 45

5.1 Experimental setup 45

5.1.1 Test bed and scale *45*

5.1.2 Metrics *46*

5.1.3 Baseline system *47*

5.2 Functional and reliability testing 47

5.3 Messaging latency under load 49

5.3.1 Scenario *49*

5.3.2 Results *49*

5.4 Mesh versus SFU 51

5.4.1 Experiment design *51*

5.4.2 Measurement pitfalls *52*

5.4.3 Results: bandwidth *53*

5.4.4 Results: CPU *56*

5.4.5 Results: delay and quality *57*

5.4.6 Effect of simulcast and layer selection *59*

5.4.7 Discussion *60*

5.5 Comparison with existing systems 61

5.6 Requirement verification....62

5.7 Limitations of the measurements....63

5.8 Chapter summary....64

CHAPTER 6. CONCLUSION AND FUTURE WORK....65

6.1 Achieved results against the objectives....65

6.2 Contributions....66

6.3 Limitations....67

6.4 Future work....67

REFERENCES....69

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ACK	Acknowledgement
API	Application Programming Interface
CDP	Chrome DevTools Protocol
CPU	Central Processing Unit
DTLS	Datagram Transport Layer Security
FPS	Frames per second
GCC	Google Congestion Control
ICE	Interactive Connectivity Establishment
JWT	JSON Web Token
MCU	Multipoint Control Unit
NACK	Negative Acknowledgement
NAT	Network Address Translation
P2P	Peer-to-peer
PLI	Picture Loss Indication
QoS	Quality of Service
RBAC	Role-Based Access Control
REST	Representational State Transfer
RTCP	RTP Control Protocol
RTP	Real-time Transport Protocol
RTT	Round-trip time
SDP	Session Description Protocol
SFU	Selective Forwarding Unit
SPA	Single-Page Application
SRTP	Secure Real-time Transport Protocol
SSRC	Synchronisation Source identifier
STUN	Session Traversal Utilities for NAT
TTL	Time to live
TURN	Traversal Using Relays around NAT
UDP / TCP	User Datagram Protocol / Transmission Control Protocol
VP8	Video codec defined in RFC 6386
WebRTC	Web Real-Time Communication

LIST OF FIGURES

Figure 2.1. ICE candidate types and connectivity paths between a browser and the SFU 9
Figure 2.2. Mesh, MCU and SFU topologies for a room with four participants 11
Figure 3.1. Main use cases of the system 17
Figure 3.2. System block diagram 18
Figure 3.3. Core entities of the database 22
Figure 3.4. Sequence diagram of reliable message delivery 25
Figure 3.5. Sequence diagram of joining a video channel 26
Figure 3.6. Horizontal scaling: shared messaging, room affinity for media 27
Figure 4.1. Per-event authorisation pipeline of the Socket.IO gateway 29
Figure 4.2. Flowchart of idempotent message sending on the server 31
Figure 4.3. Presence heartbeat executed by each API process every 15 seconds 33
Figure 4.4. State diagram of ICE recovery on the client 36
Figure 4.5. Demand-driven video grid: from tile layout to SFU requests 38
Figure 4.6. Home page with assigned tasks, upcoming meetings and notifications 40
Figure 4.7. Workspace view with the channel list and a text channel 40
Figure 4.8. Video channel with the moderation panel and layout switch 41
Figure 4.9. Workspace administration page 42
Figure 4.10. Chat (left) and meeting (right) views on a 375-pixel-wide phone screen 43
Figure 4.11. Production deployment with Docker Compose 44
Figure 5.1. Test bed of the media experiments 46
Figure 5.2. Receipt latency percentiles at 50, 100 and 200 sockets 50
Figure 5.3. Uplink per client against the number of participants 54
Figure 5.4. Downlink per client against the number of participants 55
Figure 5.5. Browser CPU per client against the number of participants 56
Figure 5.6. Round-trip time against the number of participants (logarithmic scale) 57
Figure 5.7. Received frame rate against the number of participants 58
Figure 5.8. Downlink per client for the three SFU configurations at N = 6 and N = 10 (single encoding was measured up to N = 8) 60

LIST OF TABLES

Table 1.1. Technologies used in the system 4
Table 2.1. Techniques for server-to-client push 6
Table 2.2. Cost of the conferencing architectures for N participants 12
Table 2.3. Main concepts of mediasoup 13
Table 3.1. Actors of the system 15
Table 3.2. Functional requirements 16
Table 3.3. Non-functional requirements 16
Table 3.4. Separation of control, media and data traffic 19
Table 3.5. Database tables grouped by domain 21
Table 3.6. Workspace-level permissions 23
Table 3.7. Meeting-level permissions 23
Table 3.8. Socket.IO event catalogue 24
Table 4.1. Structure of the source code 28
Table 4.2. Implementation of presence and typing 33
Table 5.1. Test environment 45
Table 5.2. Automated test suites 47
Table 5.3. Message latency in milliseconds (two runs per level) 49
Table 5.4. Resources during the messaging load test (maximum observed) 51
Table 5.5. Configuration of the compared architectures 51
Table 5.6. Results of the mesh baseline (mean ± SD over runs) 53
Table 5.7. Results of the SFU with simulcast (mean ± SD over runs) 53
Table 5.8. Server-side cost of the SFU (mean over runs) 57
Table 5.9. SFU variants: simulcast with the default policy, single encoding, and layer 0 forced (mean over runs) 59
Table 5.10. Qualitative comparison with existing platforms 61
Table 5.11. Verification of the non-functional requirements 62
Table 6.1. Objectives of Chapter 1 and the level achieved 65

CHAPTER 1. INTRODUCTION AND OVERVIEW

1.1 Reasons for choosing the topic

Remote work, online learning and online communities have become everyday activities. A typical team, however, still uses several platforms side by side: one application for daily chat, another for video meetings and a third one for assigning tasks. Each switch between tools breaks the flow of work. The context of a discussion does not follow the team into the meeting, "who is available right now" is not visible next to the channel where the discussion happens, and the history of a project ends up scattered across services.

Platforms such as Discord and Slack have shown that organising communication as **workspaces (servers) containing channels** solves the organisational part of the problem well [1], [2]. A department or a class becomes a workspace, each topic or activity becomes a channel, and users can move between text channels and voice/video channels without leaving the application. Building such a platform, however, requires solving two technically demanding problems at the same time: low-latency, reliable delivery of messages and presence to many concurrent connections, and multipoint audio/video conferencing inside the browser that does not overload the participants' devices as the room grows. These two problems are the core of real-time web engineering and make the topic suitable for a graduation thesis that combines software design with quantitative evaluation.

1.2 Problem statement

The thesis addresses three concrete problems:

1. **Fragmented communication tools.** Messaging and video calls live in separate applications, so the working context is split.
2. **Limits of full-mesh peer-to-peer conferencing.** When every participant connects directly to every other participant, each browser has to encode and upload a separate stream for each peer. Uplink bandwidth and CPU

usage grow linearly with the number of participants, and the browser becomes overloaded as soon as several people turn on their cameras.

3. **Instant state synchronisation.** Presence, typing indicators and new messages must reach exactly the right users of the right channel with low latency. The system must also tolerate users with several open tabs, sudden network loss, and a server that runs as several processes.

1.3 Objectives

The thesis sets four objectives, which are revisited in Chapter 6:

- **O1 – Integrated workspace.** Build an "all-in-one" web platform organised as *Workspace → Channel* that lets users move freely between workspaces and between text and voice/video channels.
- **O2 – Real-time messaging and presence.** Provide channel-based real-time messaging that neither loses nor duplicates messages under retries and reconnections, together with synchronised presence (online, offline, typing).
- **O3 – Multipoint video conferencing.** Provide in-browser multipoint video conferencing without plug-ins, based on an SFU so that a client's uplink and CPU do not grow with the number of participants, together with meeting moderation.
- **O4 – Quantitative evaluation.** Measure transmission quality (delay, jitter, packet loss), the load behaviour of the WebSocket core, and compare the SFU with a mesh baseline under identical conditions.

1.4 Scope

1.4.1 Target users

Distributed project teams, educational organisations that use e-learning, and online communities.

1.4.2 In scope

- A single-page web application for modern browsers (Chromium, Firefox, Safari).
- A two-level hierarchy of workspaces and channels, with role-based access control at workspace level (owner/admin/member) and at meeting level (host/co-host/member).
- Text channels: messages, threaded replies, mentions, reactions, pins, bookmarks, attachments and search.
- Voice/video channels: join and leave, microphone/camera control, screen sharing, a dynamic video grid and moderation (waiting room, lock, admit, remove, mute, raise hand, end).
- Quality monitoring: collecting and exporting RTT, jitter, packet loss and bitrate of a call.

The system also contains a few supporting features: a Kanban task board, Google sign-in and a personal profile. They are described in the implementation chapter but are not the focus of the evaluation.

1.4.3 Out of scope

Native mobile applications, call recording, end-to-end encryption through the SFU, and multi-region deployment are outside the scope of this thesis.

1.5 Methodology

The work followed four steps:

1. **Literature study** of the WebSocket protocol, the WebRTC protocol suite (SDP, ICE/STUN/TURN, DTLS-SRTP, RTP/RTCP) and the Mesh, MCU and SFU conferencing architectures.
2. **Analysis and design**: functional and non-functional requirements, a system block diagram with three main blocks (client, application/signalling server, SFU media server), the database and the access-control model.

3. **Implementation**: a Node.js + Socket.IO + PostgreSQL server, a mediasoup SFU with a worker pool, and a React/TypeScript single-page client with a dynamic video grid.
4. **Experiments and evaluation**: automated tests, a WebSocket load test with 50/100/200 connections, and a media laboratory in which each participant is a real headless Chromium browser with a synthetic camera. The laboratory measures uplink, downlink, CPU, RTT, jitter and packet loss for a mesh baseline and for the SFU.

1.6 Technologies used

Table 1.1 lists the main technologies of each layer. The client is built with React [3], the server with Node.js and Socket.IO [4], and the media server with mediasoup [5].

Table 1.1. Technologies used in the system

Layer	Technologies
User interface	React 19, TypeScript 5, Vite 6, mediasoup-client 3, socket.io-client 4
Application server	Node.js 22, TypeScript, Socket.IO 4, JWT (jose), scrypt password hashing
Media server	mediasoup 3 (C++ workers), VP8 simulcast, Opus
Data	PostgreSQL 17 (data, Socket.IO adapter, presence with TTL); file storage in the database or S3
Infrastructure	Docker Compose, Nginx (HTTPS, reverse proxy), coturn (STUN/TURN)
Testing	node:test, Socket.IO load scripts, media laboratory (Chromium + Chrome DevTools Protocol)

1.7 Report structure

The rest of the report is organised as follows:

- **Chapter 2** summarises the theory needed for the project: real-time web communication, WebRTC and multipoint conferencing architectures.

- **Chapter 3** presents the requirements, the system block diagram and the analysis of each block, the database, access control and the real-time protocols.
- **Chapter 4** describes the implementation through algorithm diagrams and the key parts of the source code.
- **Chapter 5** presents the experiments and evaluates the system against a mesh baseline and against the requirements.
- **Chapter 6** concludes the thesis, reviews the objectives of this chapter and proposes future work.

CHAPTER 2. THEORETICAL BACKGROUND

This chapter summarises only the theory that the project relies on: real-time communication on the web for messaging and presence, WebRTC for media, and the three architectures for multipoint conferencing. The comparison at the end of the chapter motivates the architecture of Chapter 3 and the experiments of Chapter 5.

2.1 Real-time web communication

2.1.1 From request–response to WebSocket

HTTP is a request–response protocol: the server only answers when the browser asks. To show a new message or a "typing" indicator as soon as it happens, older techniques simulate server push, each with its own limitation (Table 2.1).

Table 2.1. Techniques for server-to-client push

Technique	How it works	Limitation
Short polling	The client asks periodically, e.g. every 2 s	Delay up to one polling period; many empty requests
Long polling	The server holds the request until data is available	One new HTTP round trip and full headers per event
Server-Sent Events	One-way HTTP stream from server to client	No client-to-server channel on the same connection
WebSocket	One long-lived, full-duplex TCP connection	Connection state, reconnection and scaling must be managed

WebSocket [6] starts with an HTTP/1.1 `Upgrade` handshake. The client sends a random `Sec-WebSocket-Key`; the server proves that it understands the protocol by returning the base64-encoded SHA-1 hash of that key concatenated with a fixed GUID, with status `101 Switching Protocols`. After the handshake the TCP connection carries WebSocket frames instead of HTTP. Each frame has a 2–14 byte header (FIN bit, opcode, mask bit and payload length); client-to-server frames are masked with a 32-bit key to protect intermediary caches, and ping/pong control frames detect dead connections. Because headers are sent only once, WebSocket is

well suited to chat, typing and presence, which all need low-latency, two-way traffic. The system therefore uses WebSocket for every real-time interaction.

2.1.2 Socket.IO

Socket.IO [4] is a library on top of WebSocket that adds the features an application needs:

- **Named events with acknowledgements.** The server can return a result to exactly the request that emitted an event, through a callback passed to `socket.emit()`. The system uses acknowledgements for every write operation (sending a message, WebRTC signalling), so the client knows whether the operation succeeded.
- **Rooms.** The server groups sockets into rooms, for example per user (`user:<id>`) or per meeting (`meeting:<id>`), and emits events to a room. This is the mechanism used to deliver messages to the right audience.
- **Middleware.** Every connection must present an access token, and middleware can inspect every event before it reaches its handler.
- **Automatic reconnection and adapters.** When several server processes run, an adapter forwards `io.to(room).emit(...)` calls to the other processes, so a client connected to process A still receives events produced on process B.

2.1.3 Presence synchronisation

Presence answers the question "who is online" over a constantly changing set of connections. Three difficulties must be handled: a user may have several connections (tabs, devices), so the user is offline only when the last connection closes; connections can die without notice (network loss, a killed server process), so each presence record needs a time-to-live (TTL) renewed by a heartbeat; and presence changes should only be sent to the users who share a workspace with that user, so the number of messages is proportional to the number of related users rather than to the whole system. Typing indicators are short-lived: the client sends `typing: true` when

the user starts typing and `false` after a few seconds of inactivity; the server only forwards them.

2.2 WebRTC

WebRTC [7] is a set of browser APIs and IETF protocols for sending audio, video and data between endpoints without plug-ins. The main APIs are `getUserMedia()`/`getDisplayMedia()` for capturing camera, microphone and screen, `RTCPeerConnection` for establishing a session, `RTCRtpSender`/`RTCRtpReceiver` for configuring encodings, and `getStats()` for standardised statistics [8], which are the source of the QoS measurements in Chapter 5. WebRTC deliberately does not define signalling: the application must exchange session descriptions itself. In this system signalling travels over Socket.IO.

2.2.1 Session negotiation with SDP

A WebRTC session is described in SDP [9] and negotiated with the offer/answer model [10] as specified by JSEP [11]. The description lists one media section per track, the codecs and their payload types (`a=rtpmap`), the ICE credentials and candidates, the DTLS certificate fingerprint, and options such as `a=group:BUNDLE` (all tracks share one transport), `a=rtcp-mux` (RTP and RTCP share one port) and `a=simulcast` (several encodings of one track). With the mediasoup SFU the browser does not negotiate with another browser: the `mediasoup-client` library builds the local SDP from parameters that the server sends through signalling (ICE parameters, candidates, DTLS parameters and RTP parameters), and the server never parses SDP text.

2.2.2 NAT traversal: ICE, STUN and TURN

Most devices are behind NAT and have no public address. ICE [12] solves this in three steps (Figure 2.1). First, each endpoint gathers **candidates**: *host* addresses of its own interfaces, *server-reflexive* addresses learned by asking a STUN server [13], and *relay* addresses allocated on a TURN server [14]. Second, the endpoints pair their

candidates and run connectivity checks (STUN Binding requests) on each pair in priority order; host candidates have the highest priority and relay candidates the lowest, so ICE always prefers a direct path. Third, the controlling endpoint nominates one successful pair, and media flows on it. TURN relays all media and consumes server bandwidth, so it is only used when a direct path is impossible (symmetric NAT, UDP blocked).

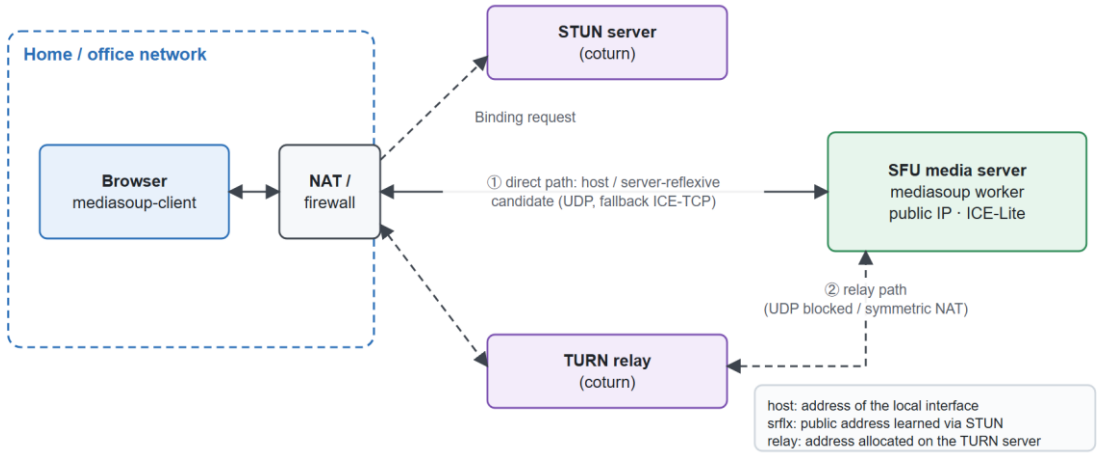


Figure 2.1. ICE candidate types and connectivity paths between a browser and the SFU

A media server with a fixed public address, such as mediasoup, runs **ICE-Lite**: it only announces its own host candidates and answers checks, while the browser controls the process. The server listens on both UDP and TCP, so media can still flow over ICE-TCP when UDP is blocked. To avoid distributing a permanent TURN password, the system uses time-limited TURN credentials [15]: the username is `<expiry timestamp>:<userId>` and the password is `base64(HMAC-SHA1(secret, username))`, which the TURN server (coturn) verifies with the same shared secret.

2.2.3 Security: DTLS-SRTP

WebRTC mandates encryption. After ICE has found a path, the endpoints run a DTLS handshake (TLS over UDP) and compare the peer's certificate with the fingerprint received through signalling; since signalling runs over secure WebSocket, the fingerprint cannot be forged. Keys derived from DTLS [16] are then used by SRTP [17] to encrypt and authenticate every RTP and RTCP packet. With an SFU

this protection is hop-by-hop (client ↔ SFU): the SFU decrypts packets to read RTP headers and re-encrypts them for each receiver.

2.2.4 RTP, RTCP and QoS metrics

RTP [18] carries media. Its 12-byte fixed header contains a payload type, a 16-bit **sequence number** (to detect loss and reordering), a 32-bit **timestamp** in codec clock units and a 32-bit **SSRC** that identifies the stream. RTCP is the companion control channel: Sender and Receiver Reports carry packet counts, cumulative loss, the highest sequence number and the interarrival jitter; feedback messages carry NACK (retransmission requests), PLI (requests for a new key frame) and transport-wide congestion control feedback. The QoS metrics used in this thesis are:

- **Jitter**, the variation of packet transit time. With $D(i-1,i) = (R_i - R_{i-1}) - (S_i - S_{i-1})$, where R is the arrival time and S the RTP timestamp, the smoothed estimate is $J_i = J_{i-1} + (|D(i-1,i)| - J_{i-1})/16$.
- **Round-trip time (RTT)**, measured from RTCP reports or, in the browser, from STUN checks on the selected ICE candidate pair.
- **Packet loss ratio** in a measurement window: $\Delta lost \;/\; (\Delta received + \Delta lost)$.
- **Jitter-buffer delay**, the average time a frame waits in the receive buffer before decoding, which is the main component of the end-to-end delay perceived by the user.

2.2.5 Congestion control, simulcast and codecs

Network capacity is not constant, so the sender estimates it continuously. Chrome uses Google Congestion Control (GCC) [19], which treats growing queuing delay as a sign of imminent congestion and reduces the bitrate, resolution or frame rate; the browser reports the reason in `qualityLimitationReason` (`bandwidth` or `cpu`). To serve receivers with different capacities, the sender can use **simulcast**: it encodes the same video several times at different resolutions, each with its own RTP stream, and the SFU forwards the version that suits each receiver at the cost of extra

encoders and uplink on the sender. The system uses VP8 [20] simulcast with three layers *q/h/f* (quarter, half and full resolution, at most 150/400/1200 kbps) and Opus [21] for audio.

2.3 Multipoint conferencing architectures

Consider a room with N participants, each sending one video stream of bitrate b. Figure 2.2 shows the three classical architectures.

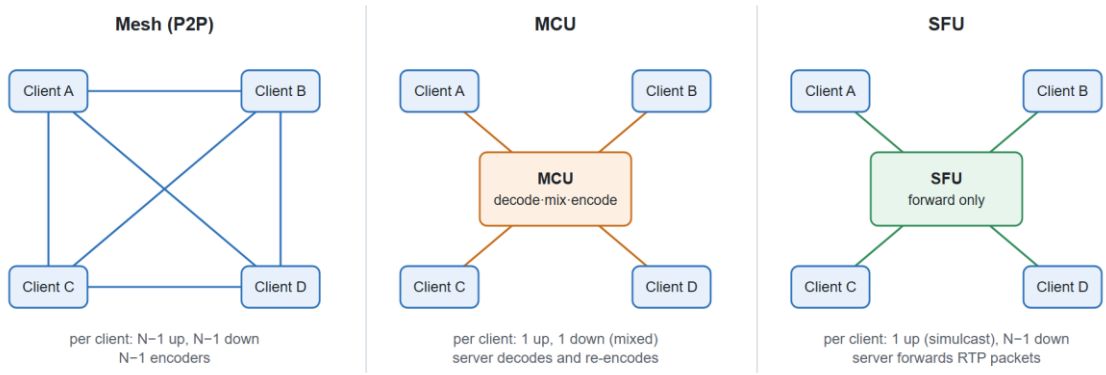


Figure 2.2. Mesh, MCU and SFU topologies for a room with four participants

2.3.1 Mesh

Every pair of participants has its own peer connection, N(N−1)/2 connections in total. Each client encodes its video N−1 times (one encoder per connection, because each connection runs its own congestion control), uploads N−1 streams and decodes N−1 streams. No media server is needed and the delay is minimal, but the uplink and CPU of every client grow linearly with N. Because the uplink is usually the weakest part of home and mobile networks, mesh is practical only for very small rooms.

2.3.2 MCU

Each client sends one stream to a Multipoint Control Unit, which decodes all streams, mixes them into one composite picture and sound, and re-encodes the result for each participant. Clients send and receive only one stream, but the server spends

a large amount of CPU on decoding, mixing and encoding, adds processing delay, imposes a fixed layout and reduces quality through transcoding.

2.3.3 SFU

Each client sends one stream (or one simulcast group) to a Selective Forwarding Unit. The SFU does not decode media: it reads RTP headers and forwards each packet only to the receivers that need it, while handling RTCP feedback for each receiver. The client's uplink is independent of N, the server's CPU cost is low because it works at packet level, and each receiver can get a different quality layer. The client's downlink still grows with N−1, which can be reduced by selecting lower layers or pausing streams that are not displayed [22].

2.3.4 Quantitative comparison

Table 2.2 compares the per-client and server costs of the architectures.

Table 2.2. Cost of the conferencing architectures for N participants

Quantity (per client)	**Mesh**	**MCU**	**SFU**	**SFU + simulcast**
Video encoders	N−1	1	1	3 (constant)
Uplink	(N−1)·b	b	b	b_q + b_h + b_f
Downlink	(N−1)·b	b_mix	(N−1)·b	sum of selected layers
Decoded streams	N−1	1	N−1	N−1 or number of visible tiles
Server CPU	none	very high	low	low
Added delay	none	decode + mix + encode	one forwarding hop	one forwarding hop

For N = 6 and b = 1.2 Mbps, a mesh client would have to upload 5 × 1.2 = 6 Mbps with five encoders, whereas an SFU client with simulcast uploads at most 0.15 + 0.4 + 1.2 ≈ 1.75 Mbps with three encoders, independently of N. Previous studies of open-source SFUs [23] report the same qualitative behaviour. The project therefore chooses an **SFU**: client uplink and CPU stay constant as N grows, the server does not

decode media, and simulcast combined with demand-driven layer selection also keeps the downlink under control. Chapter 5 verifies this choice experimentally against a mesh implementation on the same hardware.

2.4 The mediasoup SFU

mediasoup [5] is an SFU library for Node.js whose media core is written in C++ and runs in separate worker processes, while the control API is written in JavaScript/TypeScript. Its main concepts are listed in Table 2.3.

Table 2.3. Main concepts of mediasoup

Concept	Role
Worker	A C++ process that uses one CPU core; the application creates a pool of workers
Router	A media "room" with a common set of codecs; forwards packets from producers to consumers
WebRtcTransport	An ICE-Lite/DTLS/SRTP endpoint for one client (usually one for sending and one for receiving)
Producer	A track sent by a client (microphone, camera or screen)
Consumer	A track sent by the SFU to one client; can be paused, resumed and switched between simulcast layers

The minimal signalling flow is: obtain the router's RTP capabilities, create two transports, connect them with the client's DTLS parameters, produce local tracks, and consume remote tracks. Consumers are created **paused** and resumed only when the client is ready, so the first key frame is not lost. mediasoup also sends probing packets on a dedicated stream (SSRC 1234) to discover spare bandwidth before switching a consumer to a higher layer; these packets must be excluded from client-side quality metrics (Section 5.4.2).

2.5 Role-based access control

Role-based access control (RBAC) [24] assigns permissions to roles rather than to individual users. The system uses two levels of roles. At **workspace level**, the single *owner* has full control and can transfer ownership, *admins* manage members,

channels and invitations, and *members* use the channels they are allowed to see; private channels add an explicit list of channel members. At **meeting level**, the *host* and *co-hosts* moderate the room (waiting room, lock, admit, remove, mute) and only the host can appoint co-hosts or end the meeting. Every socket event that writes data and every media signalling call is checked against the current role before it reaches its handler.

2.6 Chapter summary

WebSocket and Socket.IO provide a two-way channel with acknowledgements, rooms and horizontal scaling through adapters, which is the basis for chat, typing, presence and signalling. WebRTC captures, encrypts and transports media and exposes standard statistics; ICE/STUN/TURN handle NAT traversal and RTP/RTCP carry media and quality feedback. Among the three conferencing architectures, the SFU gives the best balance between client load, server load and delay, and with simulcast it can control both uplink and downlink. These conclusions drive the design in Chapter 3.

CHAPTER 3. SYSTEM BLOCK DIAGRAM AND DESIGN

This chapter starts from the requirements, presents the block diagram of the whole system, and then analyses each block: what it contains, what it is responsible for and how it works with the other blocks. The last sections describe the database, the access-control model and the real-time protocols that connect the blocks.

3.1 Requirements analysis

3.1.1 Actors

Table 3.1 lists the actors of the system. Workspace roles and meeting roles are independent: the same person can be a member of a workspace and the host of a meeting in one of its channels.

Table 3.1. Actors of the system

Actor	Description
Guest	Not signed in; can view the landing page, register, sign in and open an invitation link
Member	Belongs to one or more workspaces; sends messages and joins voice/video channels
Admin	A member who manages channels, members and invitations of a workspace
Owner	The creator of a workspace; full control, changes roles, transfers ownership, archives
Host / co-host	Roles inside one meeting session; moderate the room

3.1.2 Functional requirements

Table 3.2 lists the functional requirements, grouped by feature. F1–F7 are the user-facing functions; F8 supports the evaluation in Chapter 5.

Table 3.2. Functional requirements

ID	Group	Requirement
F1	Account	Register with e-mail OTP verification, sign in with password or Google, refresh sessions, reset password
F2	Workspace	Create workspaces; invite members by link or e-mail; owner/admin/member roles; archive and restore
F3	Channel	Text, voice and video channels; private channels with a member list; channel ordering
F4	Messaging	Send, edit and delete messages; threaded replies, mentions, reactions, pins, bookmarks, attachments, search
F5	Presence	Online/offline status, typing indicators and unread counters in real time
F6	Conferencing	Join and leave a video channel; microphone and camera control; screen sharing; dynamic video grid
F7	Moderation	Waiting room, room lock, admit, remove, mute others, raise hand, end the meeting
F8	Measurement	Display and export call QoS (RTT, jitter, packet loss, bitrate)

3.1.3 Non-functional requirements

Table 3.3 states the non-functional requirements and how each of them is verified in Chapter 5.

Table 3.3. Non-functional requirements

Group	Requirement	Verification
Latency	Messages reach receivers in < 100 ms (95th percentile) on a local network	Load test with 50/100/200 connections (Section 5.3)
Media scalability	Client uplink and CPU do not grow with the number of participants	Mesh/SFU experiment (Section 5.4)
Media quality	Packet loss ≈ 0, jitter < 30 ms, RTT < 150 ms on a good network	`getStats()` measurements (Section 5.4)
Reliability	No lost or duplicated messages after lost acknowledgements or reconnection; automatic ICE recovery	Integration tests (Section 5.2)
Security	Hashed passwords, short-lived tokens, per-event authorisation, DTLS-SRTP	Permission tests (Section 5.2)

Group	Requirement	Verification
Horizontal scaling	Several API processes; meetings placed on workers/nodes	Two-process integration test (Section 5.2)

3.1.4 Main use cases

Figure 3.1 shows the main use cases grouped by actor. Members work inside workspaces and channels; owners and admins manage the workspace; hosts and co-hosts moderate a meeting.

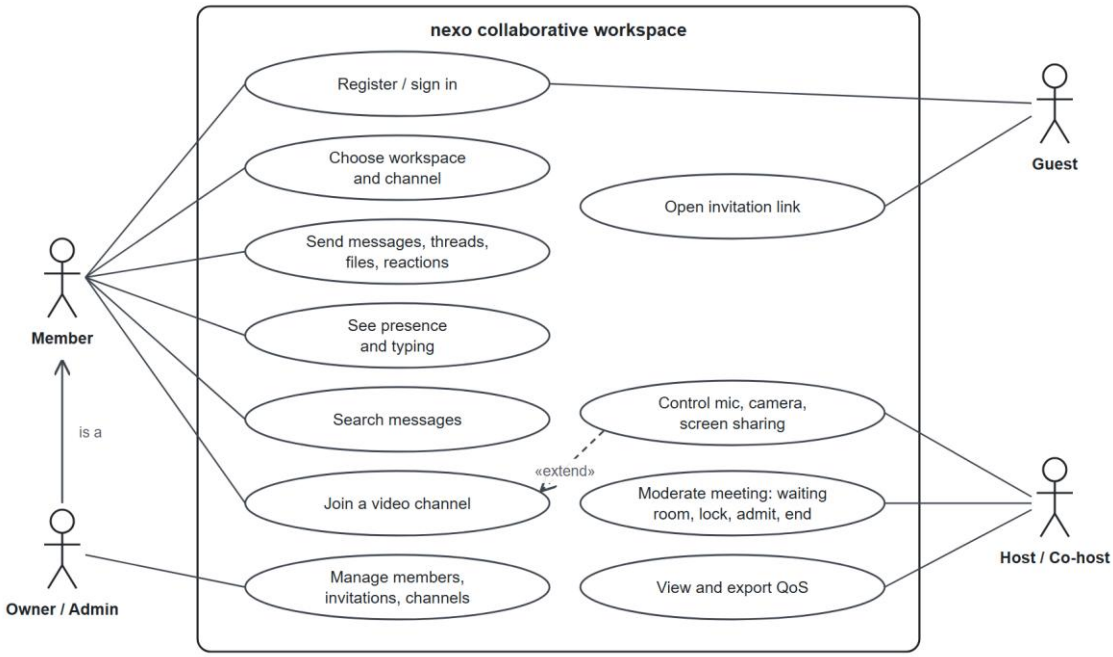


Figure 3.1. Main use cases of the system

3.2 System block diagram

3.2.1 Overview

Figure 3.2 is the block diagram of the system. It consists of three main blocks (the client, the application and signalling server, and the SFU media server) and two supporting blocks (the data block and the network traversal block).

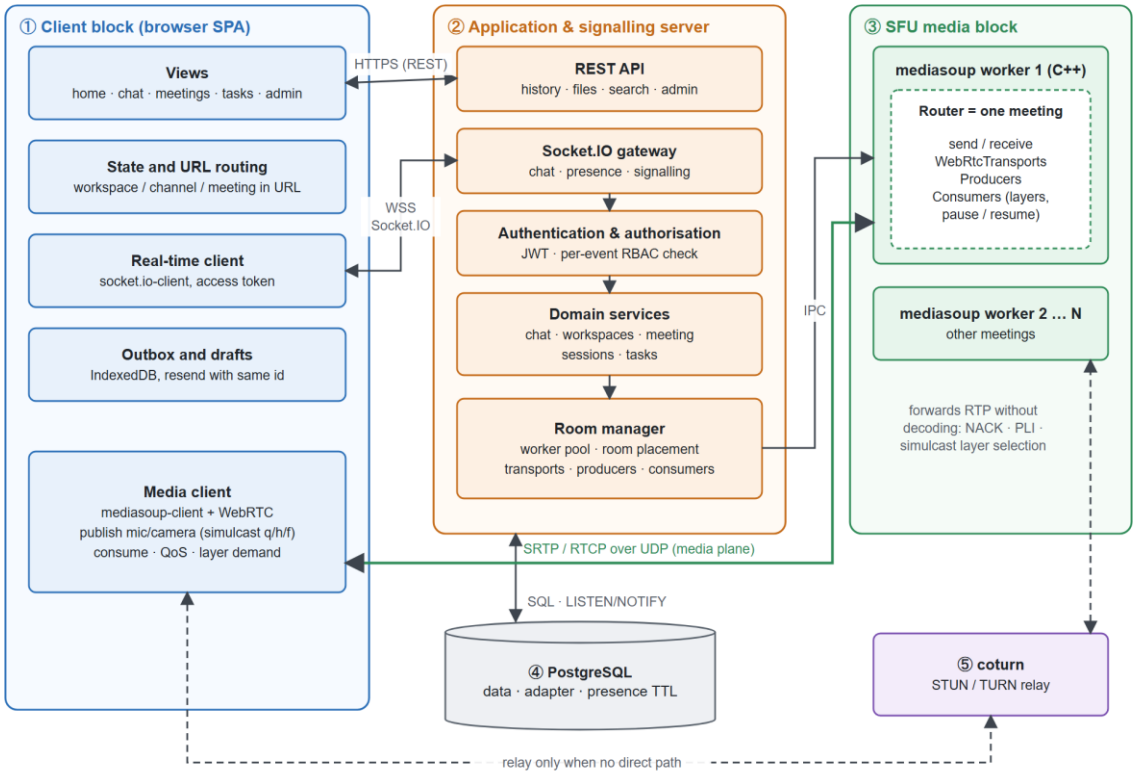


Figure 3.2. System block diagram

- The **client block** is a React single-page application running in the browser. It opens one Socket.IO connection for messaging and presence, uses REST for data that is loaded on demand, and opens two WebRTC transports (send and receive) when the user joins a video channel.
- The **application and signalling server block** is a Node.js process that exposes the REST API and the Socket.IO gateway. Every request and every event goes through authentication and authorisation before it reaches the chat, meeting or task services. The room manager in this block controls the media workers.
- The **SFU media block** is a pool of mediasoup C++ workers. Each meeting is a router placed on one worker; media flows directly between the browser and the worker.
- The **data block** is PostgreSQL. Besides business data, it stores presence with a TTL and forwards Socket.IO events between application processes.

- The **network traversal block** is a coturn STUN/TURN server used when a direct UDP path between a browser and the SFU is impossible.

3.2.2 Control plane and data plane

The most important design decision is to separate the control plane from the media plane (Table 3.4). Messages, presence and signalling are small and go through the Node.js process; audio and video packets are large and never touch Node.js but go directly between the browser and a C++ worker. As a result the Node.js process only handles small messages, and Chapter 5 shows that its CPU usage stays at about 1% of one core or less while the workers forward media for ten participants.

Table 3.4. Separation of control, media and data traffic

Traffic	Protocol	Path
Control (messages, presence, signalling)	Secure WebSocket (Socket.IO)	Browser ↔ Node.js API
Media	SRTP/SRTCP over UDP (fallback ICE-TCP, TURN)	Browser ↔ mediasoup worker
Persistent data	SQL	Node.js API ↔ PostgreSQL

3.3 Block analysis

3.3.1 Client block

The client block has five components:

1. **Views** for the home page, workspaces, channels, chat, meetings, tasks, notifications, profile and administration. The layout has a navigation rail, a channel list and a content area; on phones the columns collapse into drawers.
2. **Application state and routing.** The current workspace, channel and meeting are encoded in the URL, so reloading the page, using back/forward or sharing a link always restores the right context.

3. **Real-time client.** A single `socket.io-client` connection authenticated with a short-lived access token. It receives messages, presence and meeting events and sends signalling requests with acknowledgements.
4. **Durable outbox and drafts.** Unconfirmed messages and drafts are stored in IndexedDB, so they survive a reload and can be resent with the same identifier.
5. **Media client.** `mediasoup-client` builds the WebRTC transports from server parameters, publishes the microphone, camera (as three simulcast layers) and screen, consumes remote tracks, collects QoS statistics and decides which layer each visible tile needs.

3.3.2 Application and signalling server block

The server block runs on Node.js and contains:

1. **Authentication**: registration with e-mail OTP, password sign-in with salted scrypt hashes [25], Google sign-in, short-lived JWT access tokens [26] (15 minutes) and rotating refresh tokens stored in an HttpOnly cookie.
2. **REST API** for data loaded on demand: workspaces, channel history, files, search, notifications, tasks and administration.
3. **Socket.IO gateway** with two middleware layers: token verification on connection, and per-event rate limiting and authorisation before each handler.
4. **Domain services**: chat (messages, threads, mentions, notifications), workspaces and invitations, meeting sessions and moderation, tasks.
5. **Room manager**, which owns the pool of mediasoup workers, places each meeting on a worker and creates transports, producers and consumers on behalf of clients.

3.3.3 SFU media block

The SFU block is a pool of mediasoup workers (two by default, at most eight). Each meeting is a router on exactly one worker, chosen as the worker with the fewest

rooms. A router holds one send transport and one receive transport per participant, the producers that participants publish and the consumers that deliver remote tracks. The worker forwards RTP packets without decoding them, selects the simulcast layer for each consumer, retransmits lost packets on NACK and requests key frames when a consumer starts or changes layer. If a worker dies, only the meetings on that worker are closed; their participants are asked to rejoin and a replacement worker is created.

3.3.4 Data block

PostgreSQL stores all business data (Section 3.4). It also plays two infrastructure roles. First, the Socket.IO PostgreSQL adapter uses `LISTEN/NOTIFY` [27] to forward emitted events between API processes. Second, the `socket_presence` table stores one row per live socket with an expiry time renewed by a heartbeat, so presence is consistent across processes and stale rows of a crashed process expire automatically.

3.3.5 Network traversal block

The coturn server provides STUN and TURN. The API generates time-limited TURN credentials for each transport, so a client can fall back to a relay when UDP is blocked or the NAT is symmetric. The SFU itself announces a public (or LAN) address and accepts both UDP and TCP.

3.4 Database design

The PostgreSQL database contains 36 tables and views created by 20 versioned migrations. Table 3.5 groups the tables by domain, and Figure 3.3 shows the core entities of messaging and conferencing.

Table 3.5. Database tables grouped by domain

Domain	Main tables	Notes
Identity	`users`, `refresh_sessions`, `email_otps`, `password_resets`	Salted password hashes; refresh tokens and OTP codes stored only as hashes

Domain	Main tables	Notes
Organisation	`workspaces`, `memberships`, `channels`, `channel_memberships`, `workspace_invitations`, `audit_events`	Exactly one owner (unique index and constraint trigger); invitation tokens stored as SHA-256
Communication	`messages`, `message_attachments`, `message_reactions`, `message_mentions`, `message_pins`, `message_bookmarks`, `thread_reads`, `notifications`	Unique client message identifier per author prevents duplicates
Conferencing	`meeting_sessions`, `meeting_attendees`, `meeting_connections`, `meeting_schedules`	At most one active session per channel (partial unique index)
Operations	`socket_presence`, `api_instances`, `request_rate_limits`, `schema_migrations`	Presence TTL, process leases, rate limits
Extension	`tasks` and related tables	Kanban board (not the focus of the thesis)

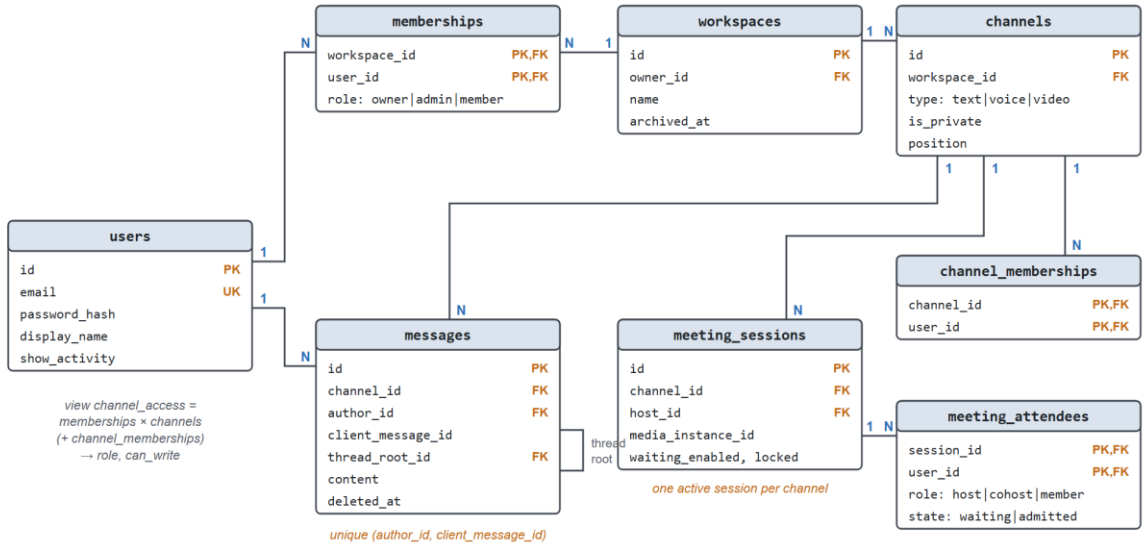


Figure 3.3. Core entities of the database

A database view, `channel_access`, joins memberships with channels (and with `channel_memberships` for private channels) and returns, for each user and channel, the role and whether the user can write (the workspace and channel are not

archived). Every read or write permission check (REST, socket, search, file download and media) uses this single view, so the access policy is defined in exactly one place.

3.5 Access control design

Table 3.6 and Table 3.7 define the permissions of the two role levels introduced in Section 2.5.

Table 3.6. Workspace-level permissions

Operation	Owner	Admin	Member	Note
Read and send in public channels	✓	✓	✓	Workspace and channel not archived
Read and send in private channels	✓*	✓*	✓*	*Requires channel membership, even for the owne*
Create, edit and reorder channels	✓	✓	–	
Invite and revoke invitations	✓	✓	–	Seven-day tokens, stored only as hashes
Change roles, transfer ownership, archive	✓	–	–	Transaction with a workspace lock
Join video channels	✓	✓	✓	Per-session limits on users, tabs and sources

Table 3.7. Meeting-level permissions

Operation	Host	Co-host	Member
Enable the waiting room, lock the room	✓	✓	–
Admit, remove, mute others	✓	✓	–
Appoint co-hosts, end the session	✓	–	–
Raise hand, control own devices	✓	✓	✓

When a user loses access, the server emits `access_revoked`, disconnects all sockets of that user on every process and closes the user's media resources. The client then reconnects and reloads data with the new permissions.

3.6 Real-time protocol design

3.6.1 Event catalogue

The Socket.IO event contract is defined once as TypeScript types in a shared package and is used by both the client and the server, so a renamed event or a changed payload is a compile-time error on both sides. Table 3.8 summarises the events.

Table 3.8. Socket.IO event catalogue

Group	Client → server (with acknowledgement)	Server → client
Channels	`join_channel`, `leave_channel`, `mark_channel_read`	`channel_read`
Messages	`send_message`, `edit_message`, `delete_message`, `react_message`	`new_message`, `message_changed`
Presence	`typing`	`user_typing`, `user_presence`, `online_users`
Meetings	`join_meeting`, `leave_meeting`, `meeting_state`, `meeting_action`	`meeting_state_changed`, `meeting_closed`, `meeting_mute`
SFU signalling	`create_webrtc_transport`, `connect_webrtc_transport`, `produce`, `consume`, `resume_consumer`, `pause_consumer`, `consumer_layer`, `close_producer`, `restart_ice`	`new_producer`, `producer_closed`, `peer_left`

3.6.2 Message delivery flow

The goal of the message protocol is that a message is never lost and never stored twice, even when acknowledgements are lost or the connection drops. Figure 3.4 shows the flow. The client assigns a `clientMessageId` and keeps the message in its outbox until it receives the acknowledgement. The server checks the current permission, takes a lock on (author, clientMessageId) and either inserts the message and broadcasts it, or, if the same identifier was already stored with the same content, returns the stored message without broadcasting it again. If the acknowledgement is

lost, the client resends with the same identifier; after a reconnection it fetches the messages it missed through REST with an `after` cursor.

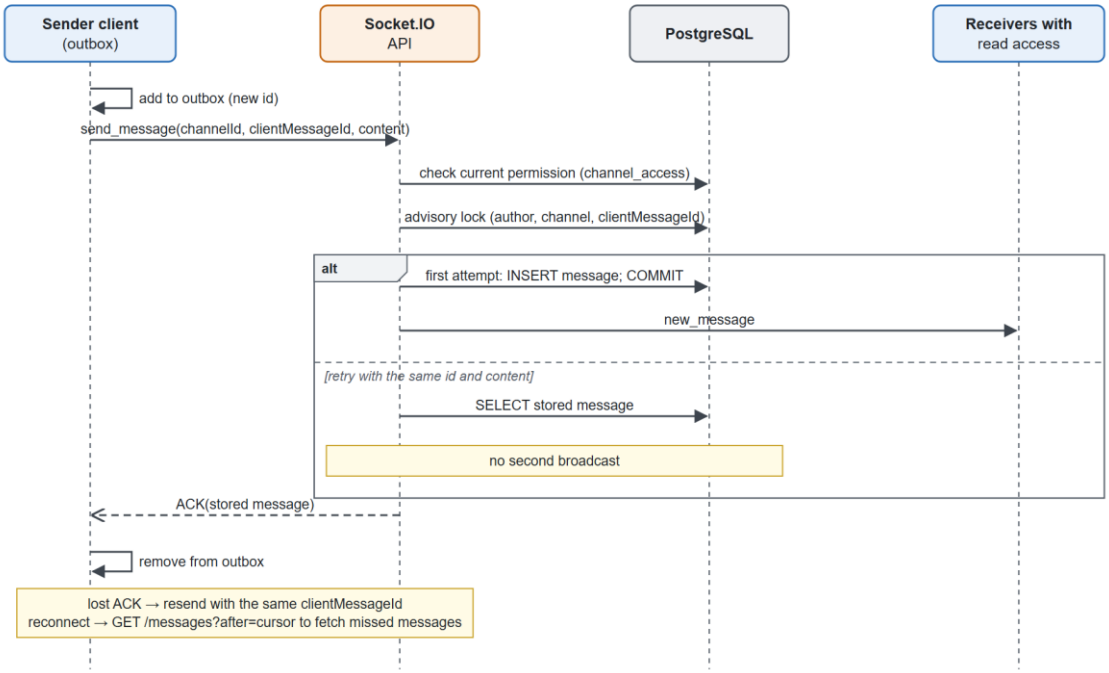


Figure 3.4. Sequence diagram of reliable message delivery

3.6.3 Joining a video channel

Figure 3.5 shows how a client joins a video channel. The server checks permissions, session limits and the waiting room, places the meeting on a worker, and returns the router's RTP capabilities. The client loads its mediasoup device, creates a send and a receive transport, publishes its tracks (the camera as three simulcast layers) and consumes the producers of the other participants. Consumers start paused and are resumed when the client is ready. Afterwards the client continuously tells the SFU which layer it needs for each tile, or pauses tiles that are not visible.

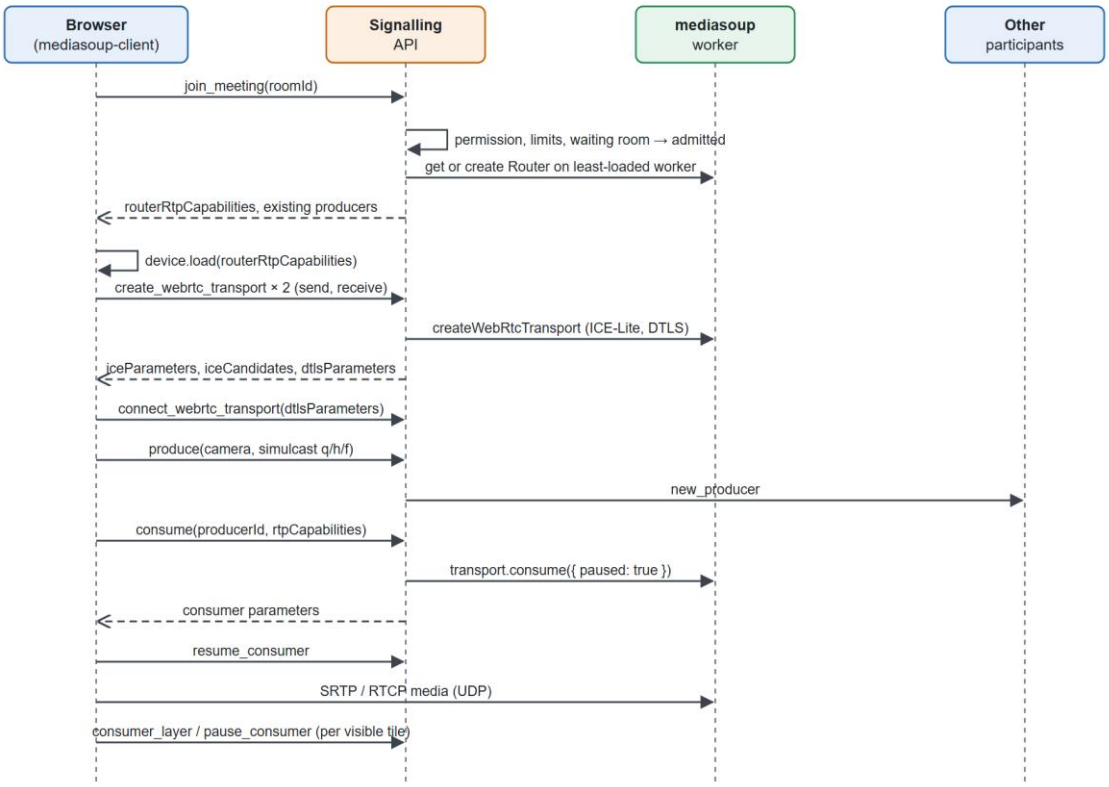


Figure 3.5. Sequence diagram of joining a video channel

3.7 Horizontal scaling design

Messaging and media scale differently, as illustrated in Figure 3.6. For messaging and presence, several API processes share PostgreSQL; the PostgreSQL adapter forwards emitted events between processes and the `socket_presence` table with a TTL and heartbeat gives a consistent view of who is online. Each process also renews a lease in `api_instances`, so the others can detect a dead process and clean up its meetings. Media cannot be shared in this way, because transports and routers live inside one worker process. A meeting therefore lives on **one** node and worker (room affinity): the session row records the media instance that owns the meeting, and clients ask the API for the media location before they open their media socket.

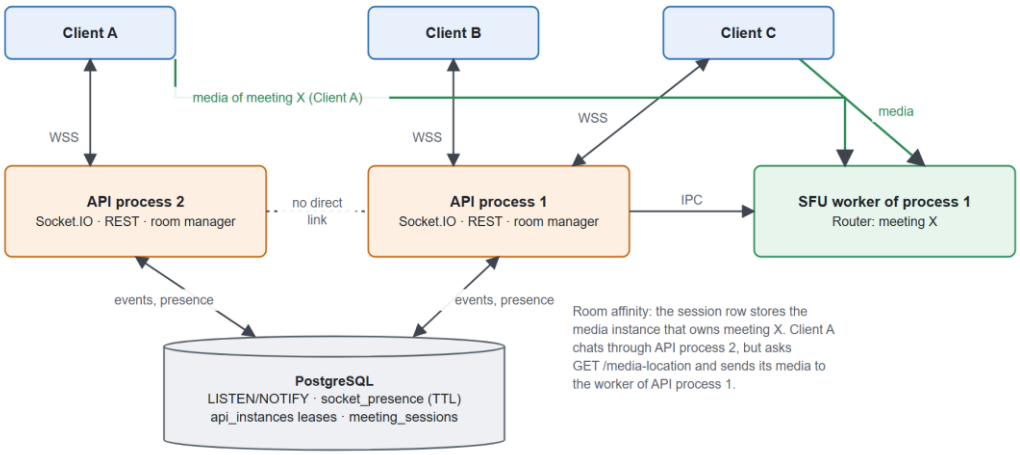


Figure 3.6. Horizontal scaling: shared messaging, room affinity for media

3.8 User interface design

The interface follows a three-column layout: a navigation rail (home, groups, meetings, tasks, notifications, settings), a list of workspaces and channels, and a content area for chat or a meeting, with an optional information panel. The video grid computes the column count that gives the largest 16:9 tiles, paginates tiles (4, 6 or 9 per page depending on the width) and offers a focus mode with one large tile. Tiles outside the current page are not rendered and their video is paused at the SFU. The connection state is always visible: the channel header shows the number of online members or "reconnecting", and messages that could not be sent are shown with *retry* and *discard* actions. Screenshots of the implemented interface are presented in Section 4.5.

3.9 Chapter summary

The system is organised as three main blocks (client, application/signalling server and SFU) and two supporting blocks (PostgreSQL and coturn). The control plane and the media plane are separated so that Node.js only handles small messages. A single database view defines access to channels, the message protocol combines client identifiers, an outbox and a uniqueness constraint to avoid loss and duplication, and meetings are pinned to one worker while messaging is shared between processes. Chapter 4 explains how these blocks are implemented.

CHAPTER 4. SYSTEM IMPLEMENTATION

This chapter describes how the blocks of Chapter 3 are implemented. Following the guidelines of the faculty, it does not reproduce the source code; it presents the algorithms as diagrams and quotes only the short pieces of code that carry the key ideas. The listings are shortened, and comments in English were added for the report.

4.1 Source code organisation

The project is an npm-workspaces monorepo written entirely in TypeScript (Table 4.1). Each feature (authentication, workspaces, chat, meetings, tasks, files, profiles) has its own routes and services on the server and its own pages, components and API client on the browser side. The Socket.IO contract (`ClientToServerEvents`, `ServerToClientEvents`) lives in the shared package, so the compiler checks both sides of every event.

Table 4.1. Structure of the source code

Folder	Content
`apps/web`	React single-page client: application providers and layout, `features/<module>`, shared UI
`services/api`	Node.js server: `modules/<domain>`, infrastructure (database, real-time, e-mail), 20 SQL migrations
`packages/shared`	Data types and the Socket.IO event contract shared by client and server
`experiments`	Mesh/SFU media laboratory and packet-capture analyser (Chapter 5)
`scripts`	WebSocket benchmark, backup/restore verification, test fixtures
`infra`	Docker Compose files for development and production, Nginx configuration

The code base contains about 17,500 lines of TypeScript/TSX and 13,000 lines of CSS in the client, about 7,000 lines of TypeScript in the server and 470 lines in the shared package.

4.2 Real-time core

4.2.1 Authentication and per-event authorisation

The Socket.IO server shares the HTTP server with the REST API and applies two middleware layers. When a socket connects, the access token from the handshake is verified and the user is loaded; connections without a valid token are rejected. Then, before **every** event, a second middleware applies the checks shown in Figure 4.1: a burst limit per socket, re-verification of the token, a rate limit of 120 write operations per minute per user, a check of the **current** channel permission in the database (not the rooms the socket joined earlier) and, for media signalling, a check that the socket is inside that meeting and has been admitted. A user in the waiting room or one who was removed therefore cannot create transports, produce or consume media.

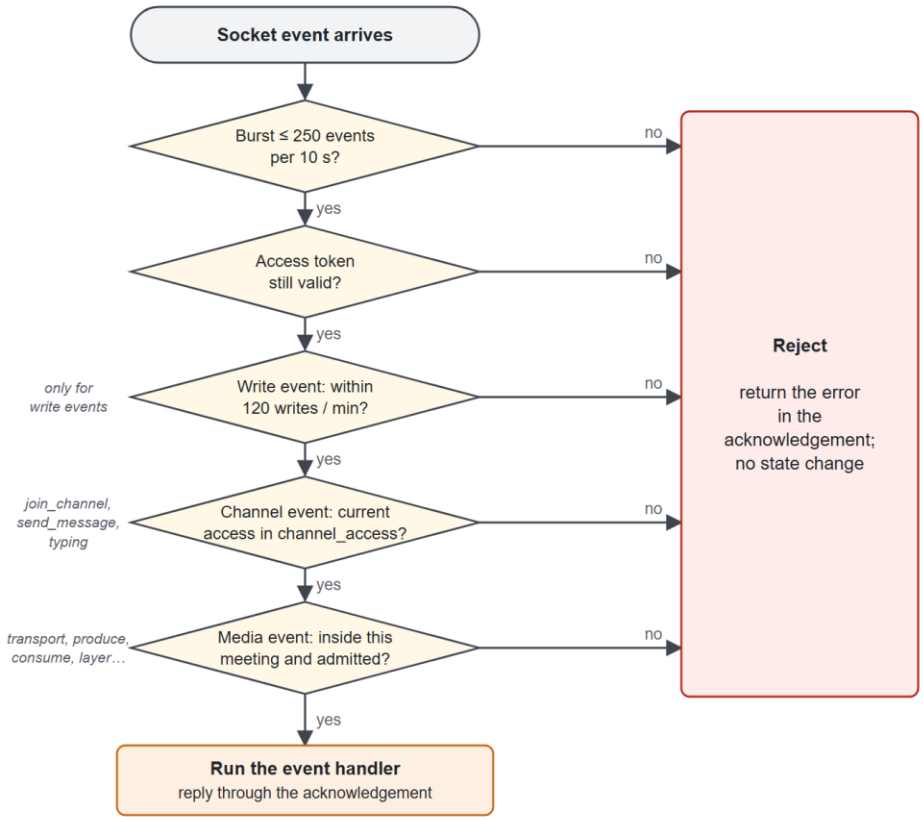


Figure 4.1. Per-event authorisation pipeline of the Socket.IO gateway

Listing 4.1 shows the core of this middleware. The event name selects which checks apply.

Listing 4.1. Per-event authorisation middleware (services/api/src/infrastructure/realtime/socket.ts, shortened)

```
socket.use(async ([event, data, ack], next) => {
  try {
    if (++burstCount > 250) throw new Error("Too many operations");
    await verifyAccessToken(token);                    // token may have expired
    if (WRITE_EVENTS.includes(event))
      await limitActor(user.id, "realtime-write", 120, 60);
    if (["join_channel", "send_message", "typing"].includes(event))
      // current rights read from the database, not from joined rooms
      await requireChannelAccess(user.id, data?.channelId);
    if (MEDIA_EVENTS.includes(event)) {
      await requireChannelAccess(user.id, data?.roomId, true);
      if (event !== "join_meeting" && activeMeetingRoomId !== data.roomId)
        throw new Error("You have not joined this room");
      if (event !== "join_meeting")
        // not in the waiting room, not removed
        await requireAdmitted(user.id, data.roomId, socket.id);
    }
    next();
  } catch (error) { reply(ack, error); }
});
```

Revocation takes effect immediately: the server emits `access_revoked` to the room `user:<id>` and disconnects all sockets of that user.

4.2.2 Exactly-once message storage

Figure 4.2 is the flowchart of `send_message`. The server validates the content (at most 4,000 characters) and the attachment, opens a transaction, locks the workspace and channel rows in share mode and re-checks the write permission inside the transaction, so a concurrent removal cannot slip in between the check and the insert. It then takes a transaction-scoped advisory lock on (author, channel, clientMessageId) and looks for an existing message with the same identifier. Three outcomes are possible: no message exists, so the message is inserted, mentions and notifications are created and `new_message` is emitted after commit; the same identifier exists with the same content fingerprint, which means the client is retrying after a lost acknowledgement, so the stored message is returned **without** a new broadcast; or the identifier exists with different content, which is reported as a conflict.

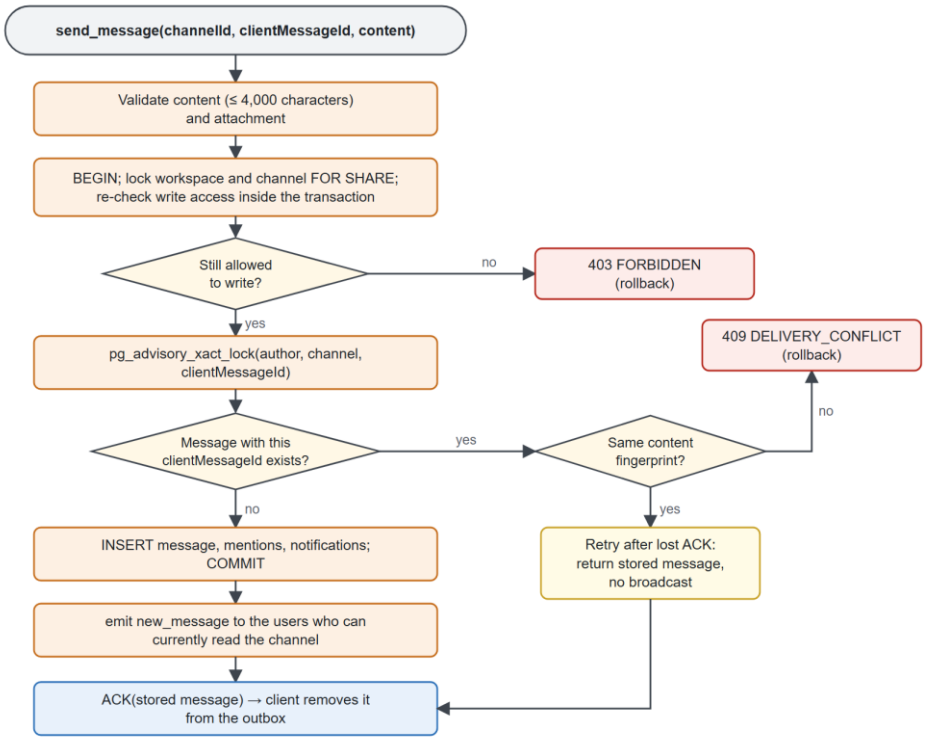


Figure 4.2. Flowchart of idempotent message sending on the server

Listing 4.2 shows the deduplication step inside the transaction. The content fingerprint (`payloadHash`) is a SHA-256 hash of the content, the reply target, the attachment, the thread root and the mentioned users, computed before the transaction starts.

Listing 4.2. Deduplication inside the send-message transaction (services/api/src/modules/workspaces/service.ts, shortened)

```
const result = await withTransaction(async (client) => {
  // Hold the current membership until the insert commits
  const access = await client.query(`SELECT m.user_id FROM memberships m ...
       AND EXISTS (SELECT 1 FROM channel_access a WHERE ... AND a.can_write)
       FOR SHARE OF m`, [channelId, userId]);
  if (!access.rowCount) throw new HttpError(403, "FORBIDDEN", "...");
  if (options.clientMessageId) {
    await client.query(`SELECT pg_advisory_xact_lock(hashtextextended($1, 0))`,
      [`${userId}:${channelId}:${options.clientMessageId}`]);
    const previous = await client.query(
      `SELECT ..., client_payload_hash AS "payloadHash" FROM messages
       WHERE channel_id=$1 AND author_id=$2 AND client_message_id=$3`,
       [channelId, userId, options.clientMessageId]);
    if (previous.rowCount) {
      if (previous.rows[0].payloadHash !== payloadHash)
        throw new HttpError(409, "DELIVERY_CONFLICT", "...");
      // retry after a lost ACK: return the stored message, no broadcast
      deduplicated = true;
      return previous;
    }
  }
  return client.query(
    `INSERT INTO messages (..., client_message_id, client_payload_hash) ...`);
});
```

New messages are emitted to the `user:<id>` rooms of the users who can **currently** read the channel, determined with the `channel_access` view, rather than to a workspace-wide room; content of a private channel therefore never reaches a member who is not in that channel. On the client, unconfirmed messages are kept in an IndexedDB outbox that survives a reload and are resent with the same `clientMessageId`. After a reconnection the client calls `GET /api/channels/:id/messages?after=<cursor>` to fetch the messages it missed and merges them by identifier. Together with the unique constraint on (author, clientMessageId), this gives *at-least-once* delivery on the network and *exactly-once* storage in the database.

4.2.3 Presence and typing

Presence is implemented with an in-memory connection counter per user and a `socket_presence` table that makes presence consistent across processes (Table 4.2). Figure 4.3 shows the heartbeat that runs every 15 seconds on each API process.

Table 4.2. Implementation of presence and typing

Component	Implementation
Connection counting	A map from user to number of sockets; online when the count goes from 0 to 1, offline when it goes from 1 to 0
Cross-process state	`socket_presence(socket_id, user_id, instance_id, expires_at)` with a 45-second TTL
Heartbeat	Every 15 s: renew the TTL of live sockets, delete expired rows, clean up meetings of dead processes, send `online_users` to each socket
Broadcast scope	`user_presence` only to workspaces the user belongs to; respects the user's "show activity" setting
Typing	`typing {channelId, isTyping}` → permission check → `user_typing` to the other members of the channel; not stored

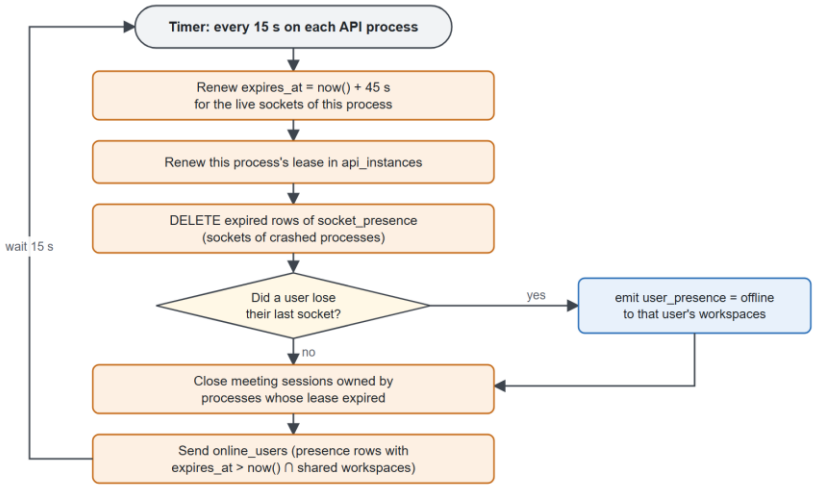


Figure 4.3. Presence heartbeat executed by each API process every 15 seconds

Because the online list is computed from rows whose `expires_at` is in the future, a user connected to process A is visible to a user connected to process B, and when a process is killed its rows expire within at most 45 seconds and its users become offline.

4.2.4 Multi-process event distribution

With `SOCKET_ADAPTER=postgres`, Socket.IO uses the PostgreSQL adapter: a call to `io.to(room).emit(...)` is published through `LISTEN/NOTIFY` and the other

processes re-emit it to their local sockets. Each process holds a lease row in `api_instances` that it renews every 15 seconds and that expires after 45 seconds. The integration test `distributed.test.js` starts two real API processes and verifies that messages, task updates and notifications cross processes.

4.3 Conferencing core

4.3.1 Worker pool and room placement

The room manager keeps a pool of mediasoup workers (`MEDIASOUP_WORKERS`, two by default, at most eight). When a meeting starts, the room manager places its router on the worker that currently hosts the fewest rooms (Listing 4.3). If a worker dies, its rooms are closed, their participants receive `media_failure` and are asked to rejoin, rooms on other workers are not affected, and the next join creates a replacement worker; the test `worker-recovery.test.js` kills a real worker to verify this behaviour.

Listing 4.3. Choosing the least-loaded worker (services/api/src/modules/meetings/media/room-manager.ts)

```
const worker = [...this.workers].sort(
  (a, b) =>
    (this.allocations.get(a.pid) ?? 0) - (this.allocations.get(b.pid) ?? 0),
)[0];
if (!worker) throw new Error("Media worker unavailable");
this.allocations.set(worker.pid, (this.allocations.get(worker.pid) ?? 0) + 1);
return worker;
```

Routers use Opus 48 kHz stereo and VP8 with a start bitrate of 1 Mbps. Each WebRTC transport enables UDP and TCP, prefers UDP and announces `MEDIASOUP_ANNOUNCED_IP`, which must be the public address in production and the LAN address (not `127.0.0.1`) in development, for the reason explained in Section 5.4.2.

4.3.2 Producer and consumer lifecycle

- `produce` checks the limit of sources per participant (by default three: microphone, camera and screen), accepts only those three kinds and emits `new_producer` to the other participants.

- `consume` checks `router.canConsume`, creates the consumer **paused** and resumes it only after the client sends `resume_consumer`, once the track is attached, so the first key frame is not lost.
- `consumer_layer` sets the preferred spatial layer of a simulcast consumer, and `pause_consumer` stops forwarding a stream that is not displayed. When the consumer is resumed, mediasoup requests a new key frame automatically.
- Closing a producer or a transport closes the dependent consumers, and `peer_left` tells the clients to remove the tile.

4.3.3 Meeting sessions and moderation

Meeting state is stored in PostgreSQL so that every process sees the same session. Each channel has at most one active session (enforced by a partial unique index), and the first person who joins becomes the host. Joining locks the session row, checks the limits on users and on tabs per user, and checks the lock and waiting-room flags; the result is *admitted* or *waiting*. A waiting participant has a socket but no SFU peer. The `meeting_action` event implements waiting room and lock (host and co-host), admit, remove and mute (host and co-host, never against the host), appoint co-host, transfer and end (host only), and raise hand (everyone). The session row also records the media instance that owns the meeting, and `GET /api/channels/:id/media-location` returns the SFU endpoint of that instance.

4.3.4 NAT traversal and ICE recovery

With every transport the server returns the list of ICE servers. If TURN is configured, it generates a time-limited credential as described in Section 2.2.2 (Listing 4.4).

Listing 4.4. Time-limited TURN credential (services/api/src/modules/meetings/media/ice.ts)

```
// valid for one hour
const username = `${Math.floor(Date.now() / 1000) + 3600}:${userId}`;
return [{
  urls,                                          // from TURN_URLS
  username,
  credential: createHmac("sha1", process.env.TURN_SHARED_SECRET)
    .update(username)
    .digest("base64"),
}];
```

On the client, every transport is watched for connection-state changes (Figure 4.4). If the transport stays *disconnected* for 5 seconds, or becomes *failed*, the client asks the server for new ICE parameters and calls `transport.restartIce()`, at most twice. If the transport is still not connected 10 seconds after a restart, the user is told to rejoin.

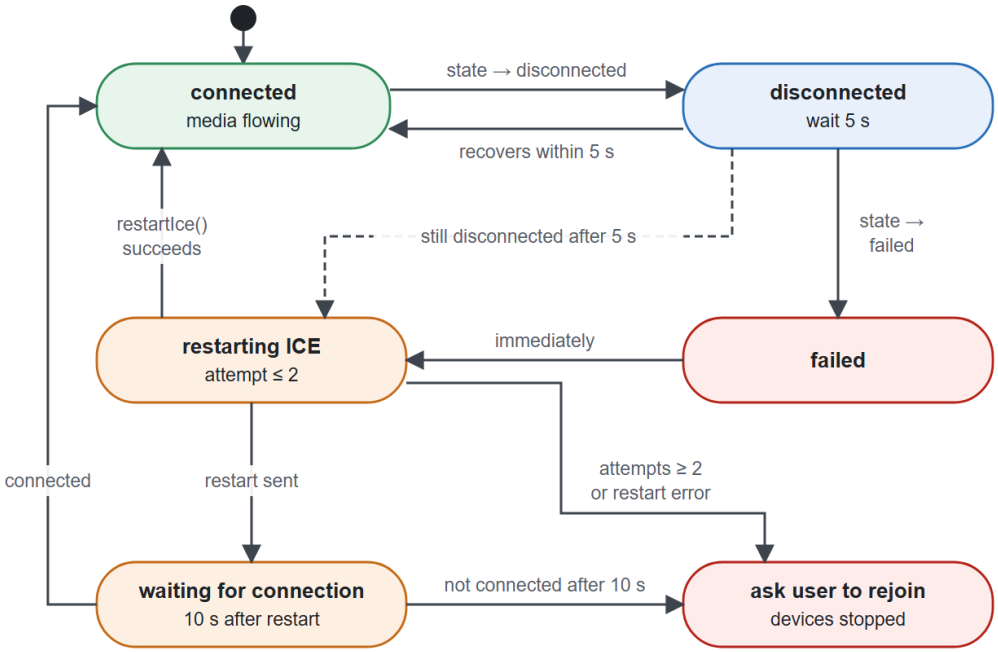


Figure 4.4. State diagram of ICE recovery on the client

4.4 Conferencing client and dynamic video grid

4.4.1 Capture and publishing

The lobby lets users preview the camera and microphone, choose devices and a capture quality (640×360, 960×540 or 1280×720) before joining. Only sources that

are switched on are published; all four microphone/camera combinations were tested to make sure that a disabled source is never sent. The camera is published with three simulcast encodings *q* (quarter resolution, 150 kbps), *h* (half, 400 kbps) and *f* (full, 1,200 kbps). Screen sharing uses `getDisplayMedia()` and is published as a separate, non-simulcast source that closes automatically when the user stops sharing from the browser bar. Muting uses `track.enabled` together with `producer.pause()`, so unmuting is immediate and does not renegotiate the transport.

4.4.2 QoS collection

A QoS collector reads `getStats()` from both transports every 2.5 seconds. RTT is taken from the selected candidate pair; jitter is the maximum over the inbound RTP streams; bitrates are computed from the change in bytes over the change in time; and packet loss is computed per window from the change in received and lost packet counters (Listing 4.5). The interface keeps the latest 300 samples (about 12.5 minutes) and can export them as CSV or JSON.

Listing 4.5. Windowed packet-loss computation (apps/web/src/features/meetings/webrtc/qos.ts, shortened)

```
if (old && sample.timestamp > old.timestamp &&
    counter.received >= old.received && counter.lost >= old.lost) {
  deltaPackets += counter.received - old.received;  // counters may be reset:
  deltaLost += Math.max(0, counter.lost - old.lost); // such windows are skipped
  hasLoss = true;
}
...
packetLoss: hasLoss && deltaPackets + deltaLost > 0
  ? (100 * deltaLost) / (deltaPackets + deltaLost)
  : null,                                            // no data is not "0% loss"
```

4.4.3 Dynamic grid and demand-driven layers

The aim of the grid is that the browser only decodes and draws what the user actually sees, at the resolution it needs. Figure 4.5 summarises the algorithm.

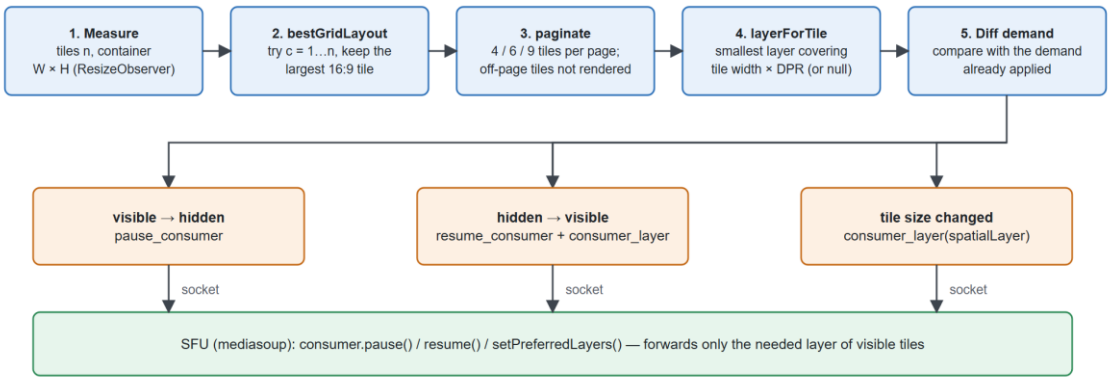


Figure 4.5. Demand-driven video grid: from tile layout to SFU requests

1. **Best layout.** For n tiles in a container of width W and height H, the algorithm tries every column count c from 1 to n, computes the number of rows r = ⌈n/c⌉ and the tile width w = min((W − gap·(c−1))/c, (H − gap·(r−1))/r · 16/9), and keeps the c that maximises w (Listing 4.6).
2. **Pagination.** A page holds 4, 6 or 9 tiles depending on the width. Tiles outside the current page are not rendered, but the audio of those participants keeps playing through separate audio elements.
3. **Layer per tile.** The smallest simulcast layer whose width still covers the tile in device pixels is selected: with a 960-pixel source, tiles up to 264 px use layer 0, up to 528 px layer 1, otherwise layer 2 (Listing 4.7).
4. **Pushing demand to the SFU.** For each participant the page computes the demand: `null` if the tile is not visible, otherwise a layer. The meeting provider compares it with the demand already applied to each consumer and sends only the differences: `pause_consumer` when a tile disappears, `resume_consumer` followed by `consumer_layer` when it reappears, and `consumer_layer` when its size changes. When the meeting page is not open (the user went to chat while staying in the call), the demand is removed and a default policy based on the number of cameras is used (more than six → layer 0, more than two → layer 1, otherwise layer 2).

Listing 4.6. Choosing the column count that gives the largest 16:9 tile (apps/web/src/features/meetings/grid-layout.ts)

```
for (let columns = 1; columns <= tiles; columns++) {
  const rows = Math.ceil(tiles / columns);
  const byWidth = (width - gap * (columns - 1)) / columns;
  const byHeight = ((height - gap * (rows - 1)) / rows) * aspect;
  const tileWidth = Math.max(0, Math.min(byWidth, byHeight));
  if (tileWidth > best.tileWidth)
    best = { columns, rows, tileWidth, tileHeight: tileWidth / aspect };
}
```

Listing 4.7. Selecting the simulcast layer for a tile (apps/web/src/features/meetings/grid-layout.ts)

```
export function layerForTile(
  cssWidth: number, devicePixelRatio = 1, sourceWidth = 960) {
  if (cssWidth <= 0) return null;          // not rendered: pause the consumer
  const needed = cssWidth * devicePixelRatio;
  if (needed <= (sourceWidth / 4) * 1.1) return 0; // q layer
  if (needed <= (sourceWidth / 2) * 1.1) return 1; // h layer
  return 2;                                        // f layer
}
```

As a result, the downlink of a client is proportional to the number and the size of the **visible** tiles rather than to the total number of participants in the room. Section 5.4.6 measures the effect of layer selection.

4.5 User interface

The client was implemented as a responsive single-page application with a Vietnamese interface. Figure 4.6 shows the home page, which prioritises the user's work: assigned tasks, upcoming meetings and notifications, with shortcuts to create a workspace or open a meeting.

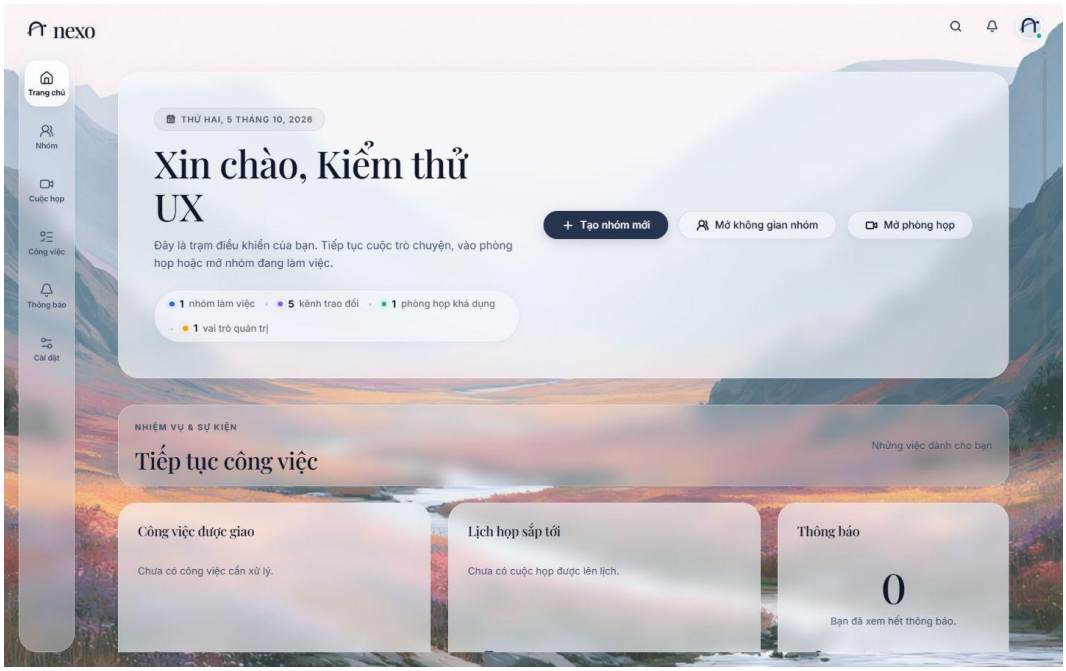


Figure 4.6. Home page with assigned tasks, upcoming meetings and notifications

Figure 4.7 shows a workspace with its channel list and an open text channel. The header shows the number of members and how many are online, the tabs switch between the overview, tasks, chat, meetings and files of the workspace, and the composer supports emoji, attachments and voice messages.

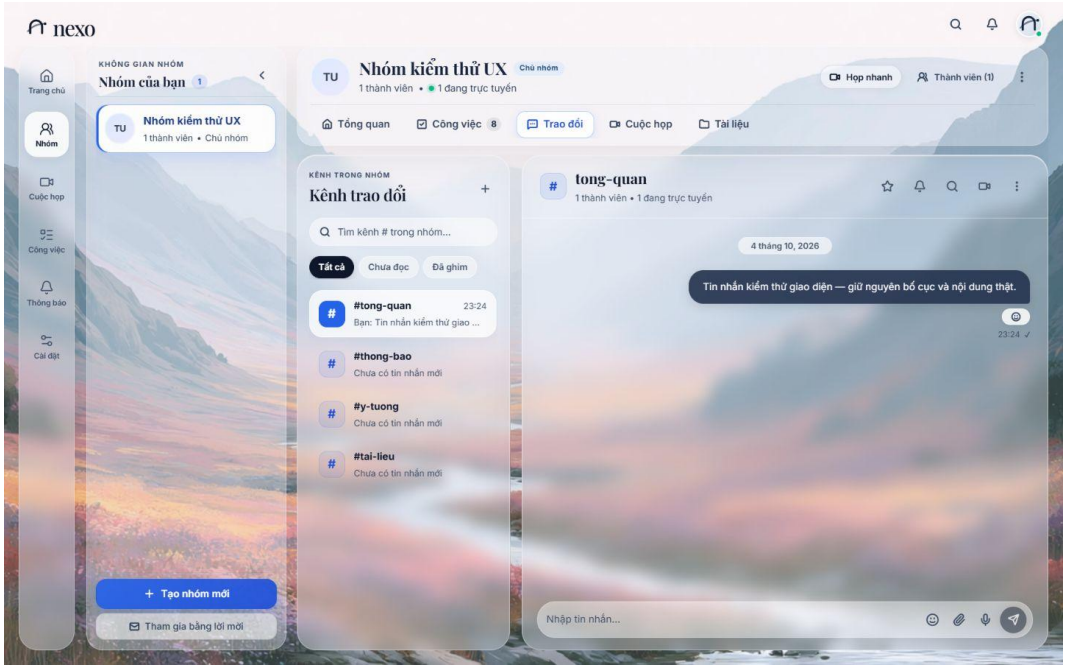


Figure 4.7. Workspace view with the channel list and a text channel

Figure 4.8 shows a video channel. The moderation panel offers the host the waiting room, room lock, "end for everyone" and raise hand, and the grid can be switched between grid and focus layouts.

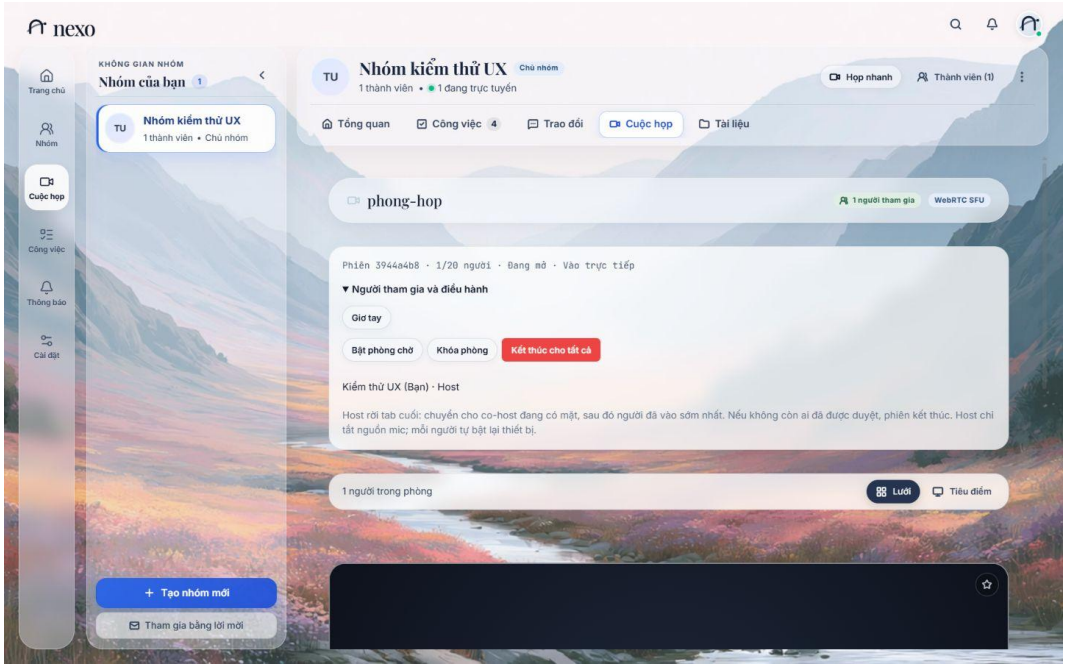


Figure 4.8. Video channel with the moderation panel and layout switch

Figure 4.9 shows the administration page, where owners and admins edit the workspace, manage members, invitations and channels, archive the workspace and read the audit log.

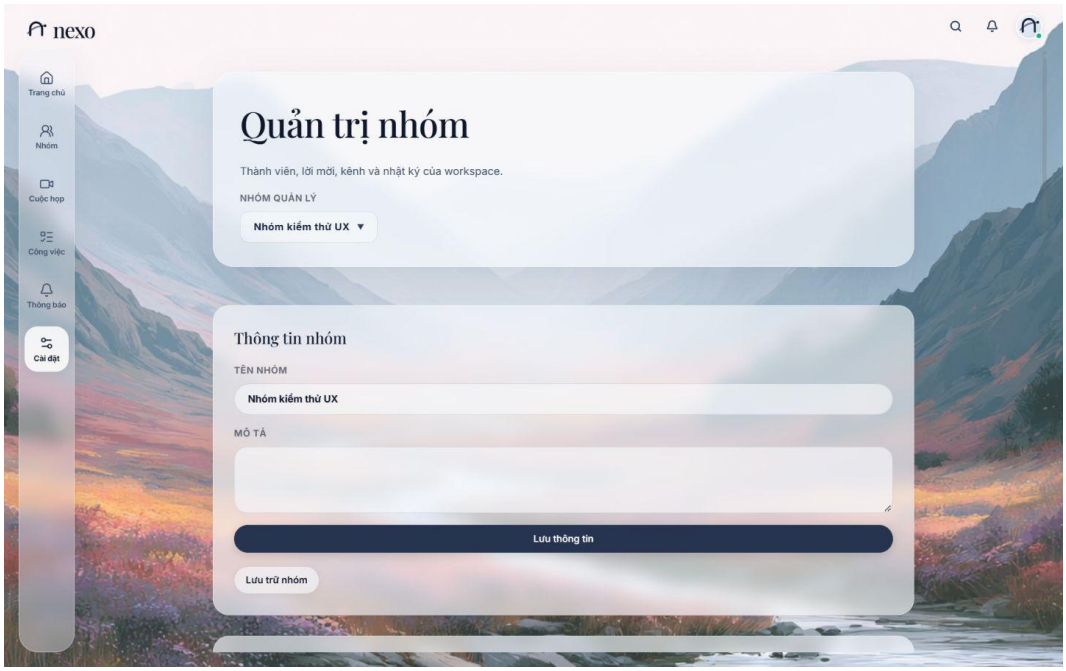


Figure 4.9. Workspace administration page

The layout adapts to tablets and phones; Figure 4.10 shows the chat and meeting views at a width of 375 pixels.

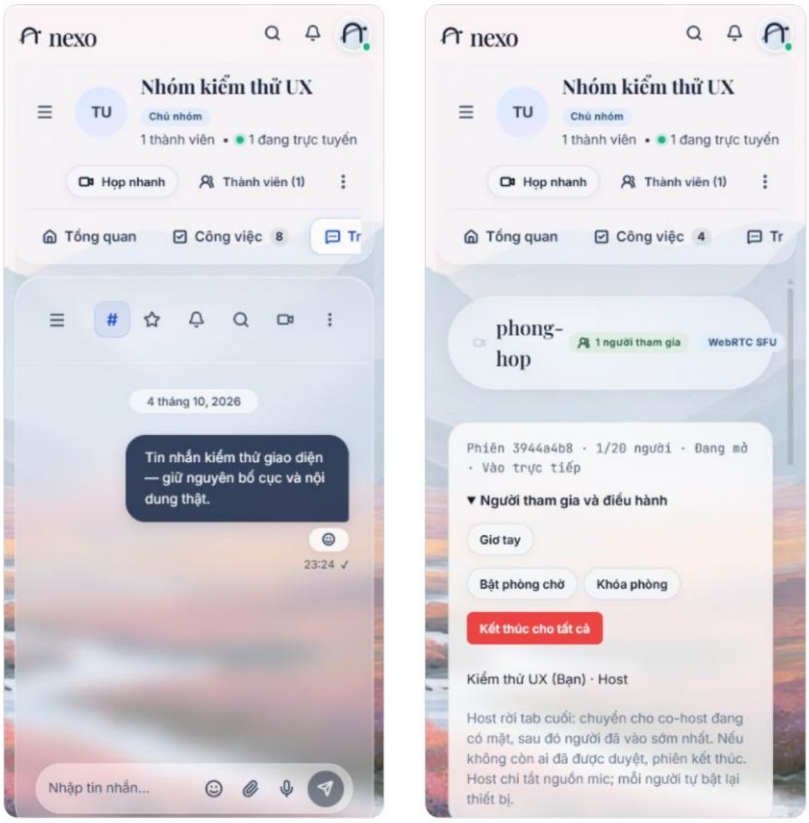


Figure 4.10. Chat (left) and meeting (right) views on a 375-pixel-wide phone screen

4.6 Deployment

The production deployment, shown in Figure 4.11, uses Docker Compose with three services: `postgres`; `api`, which runs Node.js and mediasoup and publishes the media port range 40000–40099 over UDP and TCP directly; and `web`, an Nginx server that serves the client, terminates TLS and proxies `/api` and `/socket.io` to the API. A separate coturn server provides STUN/TURN with the same shared secret as the API. The `/ready` health check succeeds only when PostgreSQL is reachable and all migrations have been applied. The environment variables for multi-process operation (`SOCKET_ADAPTER=postgres`, `API_INSTANCE_ID`, `SFU_PUBLIC_URL`) are available, but a multi-node deployment on real infrastructure has not been validated (Section 5.7).

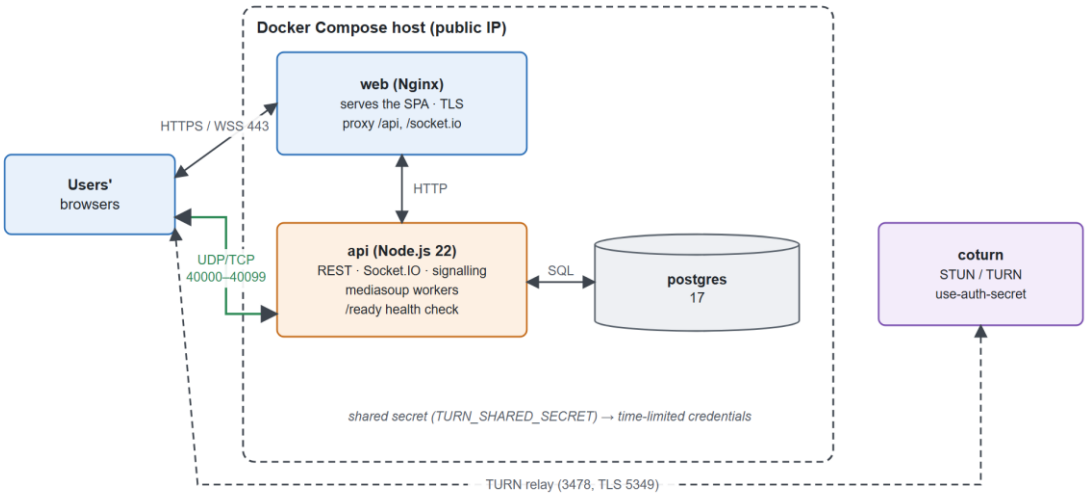


Figure 4.11. Production deployment with Docker Compose

4.7 Chapter summary

The server authorises every Socket.IO event against the current state of the database, stores each message exactly once through an advisory lock and a uniqueness constraint, and keeps presence consistent across processes with a TTL table. The conferencing core places each meeting on the least-loaded mediasoup worker, starts consumers paused, recovers ICE automatically and issues time-limited TURN credentials. The client publishes simulcast video and requests from the SFU only the layers that its visible tiles need. Chapter 5 evaluates these mechanisms.

CHAPTER 5. EXPERIMENTS AND EVALUATION

This chapter evaluates the system against the requirements of Section 3.1.3. It first describes the test bed, the metrics and the baseline system (Section 5.1), then reports functional and reliability tests (Section 5.2), the latency of the messaging core under load (Section 5.3) and the main experiment, which compares the SFU of the system with a full-mesh baseline (Section 5.4). Section 5.5 compares the system with existing platforms, Section 5.6 checks the requirements and Section 5.7 discusses the limitations of the measurements. All numbers come from raw data stored in the repository under `docs/evidence/`; values that were derived from the raw data (for example totals or ratios) are marked as *computed*.

5.1 Experimental setup

5.1.1 Test bed and scale

All experiments were run on one development machine (Table 5.1). Figure 5.1 shows how the media experiments are organised: a harness written in Node.js starts every participant as a **separate headless Chrome process** with a synthetic camera and microphone, joins it to a room, and samples its WebRTC statistics and CPU usage through the Chrome DevTools Protocol. Up to ten browsers ran at the same time for the media experiments, and up to 200 Socket.IO clients for the messaging experiment.

Table 5.1. Test environment

Component	Value
Hardware	AMD Ryzen 7 6800HS, 8 cores / 16 threads, 23.3 GiB RAM
Operating system	Windows 11 (x64)
Software	Node.js 22.19, PostgreSQL 17.11 (Docker), Google Chrome (headless), mediasoup 3
Media source	Chromium synthetic camera, 960×540 requested at 30 fps (delivers 20 fps), VP8 and Opus
Network	All browsers, the API and the SFU on the same host; media sent to the host's LAN address over UDP

Component	Value
Scale	Media: 2–10 browsers per room. Messaging: 50, 100 and 200 sockets on two API processes

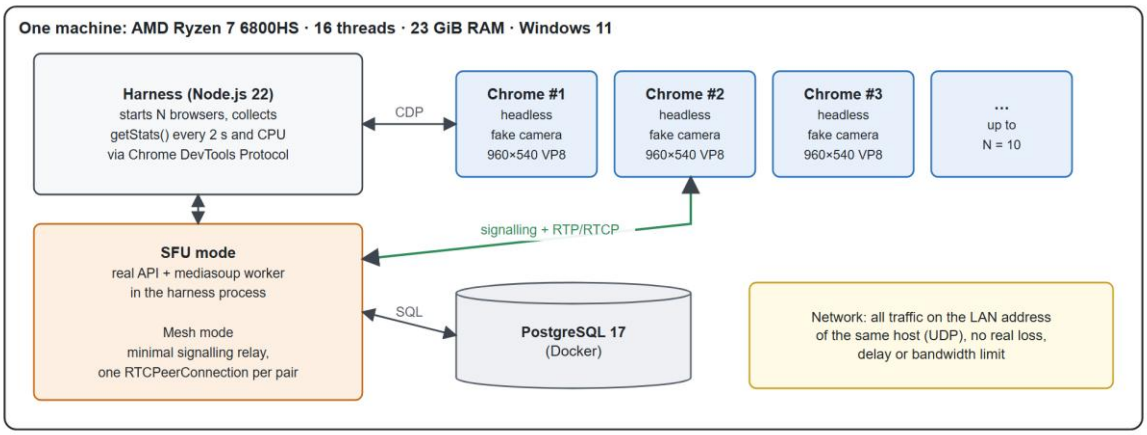


Figure 5.1. Test bed of the media experiments

Running everything on one host has a consequence that is used throughout this chapter: the network path adds no real delay, loss or bandwidth limit, so RTT, jitter and loss reflect **processing load**, not network quality, and when the total CPU demand approaches the capacity of the machine every metric degrades because of contention. Section 5.7 discusses this limitation.

5.1.2 Metrics

The media metrics are read from `getStats()` every 2 seconds and follow the definitions of Section 2.2.4 and Section 4.4.2:

- **Uplink and downlink** (kbps): change in bytes (payload and RTP headers) over the change in time.
- **Client CPU** (%): total CPU time of all processes of one browser instance; 100% equals one core.
- **RTT** (ms) of the selected ICE candidate pair; **audio and video jitter** (ms); **packet loss** (%) per window.
- **Received frame rate**, **jitter-buffer delay**, **freeze count** and the number of **active encoders** (outgoing video streams above 1 kbps).

- **Server CPU**: CPU of the mediasoup worker and of the Node.js process.

Each level of N is measured for 60 seconds after a 30-second warm-up that lets the bandwidth estimator settle, which gives 30 samples per client. The media levels were repeated **three times**; tables report the mean over runs and, where several runs exist, the sample standard deviation across runs (± SD). Message latency is reported as nearest-rank percentiles p50, p95 and p99 over all samples.

5.1.3 Baseline system

The guidelines of the faculty ask for an established reference system to compare against. For the media core, the reference is a **full-mesh peer-to-peer** implementation, the architecture that a small WebRTC application uses when it has no media server (Section 2.3.1). Implementing the baseline inside the same laboratory makes the comparison fair: both architectures use the same browser, the same synthetic source, the same codec, the same machine and the same measurement code, and differ only in the topology. The mesh client opens one `RTCPeerConnection` per pair of participants through a minimal signalling relay and sends one encoding with at most 1,200 kbps per connection; the SFU client is the production `MediasoupClientService` of nexo with three simulcast layers and the default layer policy. Section 5.5 additionally compares nexo qualitatively with existing products.

5.2 Functional and reliability testing

Table 5.2 summarises the automated tests. The integration suites run against a real PostgreSQL database and real mediasoup workers.

Table 5.2. Automated test suites

Suite	Scope	Result
Frontend unit tests (`npm test`, web)	URL identity, grid layout, QoS computation, media operations, raised hand, call-drop messages, interaction states	39 / 39 passed

Suite	Scope	Result
API unit tests (`npm test`, api)	Authentication, health, Google avatar, contracts; DB/SFU suites skipped in the default run	17 passed, 8 skipped by design
Integration tests with PostgreSQL and mediasoup (`npm run test:sfu`)	Governance and permissions, conversations, tasks, meeting sessions, storage, two API processes, worker recovery	9 / 9 passed, 0 skipped
Static checks	`typecheck`, `lint`, `build` of all workspaces	No errors

The tests verify the reliability properties required in Section 3.1.3:

- **Message idempotency.** Six concurrent requests with the same `clientMessageId` create exactly one row; dropping the acknowledgement and resending produces exactly one `new_message` broadcast; pagination with the `after` cursor never returns a message twice.
- **Revocation.** After access to a private channel is revoked, both retries and backfill after reconnection are rejected.
- **Administration.** Permissions are checked across REST, socket, search, file download, schedules and media; invitations with a wrong e-mail, expired, revoked or accepted concurrently are handled; ownership transfer works and the database refuses to delete the only owner.
- **Search.** Vietnamese text, the characters `%` and `_`, and opening a message outside the latest 100 messages are tested; a query plan on 10,000 messages confirmed that the existing timeline index is sufficient at this scale.
- **Meetings.** Waiting room, lock, removal, end and the limits on users and tabs are tested, and all four microphone/camera combinations are checked to make sure a disabled source is never published.
- **Fault tolerance.** With two workers, killing one worker does not affect participants on the other worker, and the next join creates a replacement worker.

- **Two API processes.** With the PostgreSQL adapter, messages, tasks and notifications cross processes, and revocation takes effect on both.
- **Migrations and recovery.** The 20 migrations are checksummed; two runners started at the same time on one database do not conflict; a backup restored into a fresh database preserves the schema and Unicode data.

These tests establish the correctness of control traffic and permissions; they do not measure RTP quality, which is the subject of Section 5.4.

5.3 Messaging latency under load

5.3.1 Scenario

Two API processes shared PostgreSQL through the PostgreSQL adapter (pool of eight connections per process); four workspaces with two text channels each were created. Each level (50, 100 and 200 sockets) ran for 30 seconds and was repeated twice, with **4 messages per second in total** sent by four concurrent senders. In every run the acknowledgement was dropped on purpose six times and the message was resent with the same identifier, and two messages were sent while one client was offline to exercise backfill. Most sockets are receivers, so the scenario models a busy channel rather than 200 users all typing at the same time.

5.3.2 Results

Table 5.3 and Figure 5.2 show the latencies. *ACK* is the time from emitting a message to the successful acknowledgement; *receipt* is the time from emitting a message to the moment each receiver gets `new_message`, weighted by the number of receivers.

Table 5.3. Message latency in milliseconds (two runs per level)

Sockets	ACK p50 / p95 / p99	Receipt p50 / p95 / p99	Unique messages / receipts	Errors / duplicates
50	24.99 / 39.88 / 53.79	25.01 / 32.23 / 47.02	244 / 3,052	0 / 0
100	23.69 / 46.09 / 56.81	23.78 / 34.87 / 48.07	244 / 6,096	0 / 0

Sockets	ACK p50 / p95 / p99	Receipt p50 / p95 / p99	Unique messages / receipts	Errors / duplicates
200	25.23 / 48.58 / 140.26	25.38 / 45.93 / 137.86	244 / 12,196	0 / 0

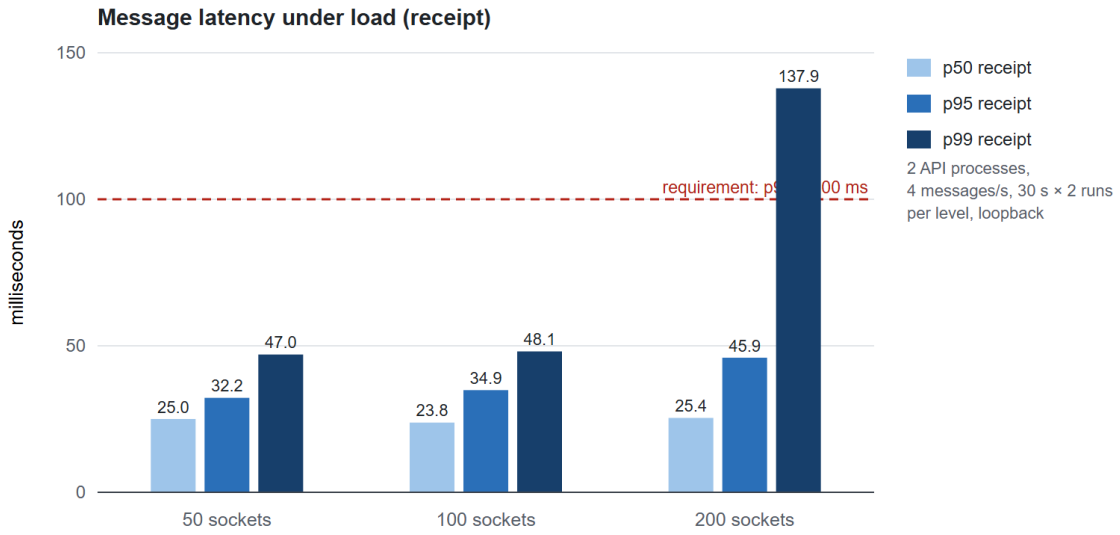


Figure 5.2. Receipt latency percentiles at 50, 100 and 200 sockets

In total 732 messages produced 21,344 receipts, and the number of stored rows equalled the number of unique messages in every run: **no message was lost or duplicated**, although 36 sends were retried after a lost acknowledgement and six clients reconnected. Each reconnection recovered the two missed messages through REST in 21.5–54.9 ms. The median latency stayed almost constant (23.7–25.4 ms) while the number of sockets grew fourfold; the 95th percentile grew from 32 to 46 ms, and only the 99th percentile at 200 sockets rose to about 140 ms.

Table 5.4 shows the resources used. Each API process used less than 6% of one core and less than 120 MiB of memory. The number of requests waiting for a database connection grew with the number of sockets (13, 31 and 120) but appeared only during the initial burst of connections and disappeared in later samples. The burst of connections and the presence updates that accompany it are therefore the first point to optimise before increasing the load, rather than simply enlarging the pool.

Table 5.4. Resources during the messaging load test (maximum observed)

Sockets	RSS per API (MiB)	CPU per API (% of one core)	Max waiting pool requests	PostgreSQL CPU / memory
50	98.8	5.60	13	5.1–5.2% / 107–112 MiB
100	95.7	3.75	31	5.0–6.0% / 111–113 MiB
200	115.2	4.66	120	4.8–5.6% / 112–113 MiB

5.4 Mesh versus SFU

This is the main experiment of the thesis. It checks the architectural choice of Section 2.3.4: as the number of participants N grows, which architecture keeps the quality of the call at an acceptable cost?

5.4.1 Experiment design

Table 5.5 summarises the configuration of the two architectures. Both are measured with the same procedure: the harness creates fresh test users and a test workspace for each level, starts N browsers, waits 30 seconds, measures for 60 seconds and removes the fixtures. Each architecture was measured three times at every level; the mesh runs stop at N = 8 because the machine was already saturated at N = 6, while the SFU runs go up to N = 10. Two additional SFU configurations isolate the effect of simulcast: a single encoding instead of three layers, and all consumers forced to the lowest layer.

Table 5.5. Configuration of the compared architectures

Item	Mesh baseline	SFU (nexo)
Topology	One RTCPeerConnection per pair, minimal signalling relay	Real API and mediasoup worker; production client code
Video encoding	One encoding, at most 1,200 kbps per connection	Simulcast q/h/f = 150/400/1,200 kbps
Receive layer	–	Default policy: more than 6 cameras → layer 0, more than 2 → layer 1, otherwise layer 2

Item	Mesh baseline	SFU (nexo)
Source	960×540 synthetic camera, 20 fps, VP8; Opus audio	Same
Rendering of received video	Off (decoded, not drawn)	Off
Levels of N	2, 4, 6, 8	2, 4, 6, 8, 10
Repetitions	3 runs × 60 s per level (30 s warm-up)	3 runs × 60 s per level (30 s warm-up)

Received video is decoded but not drawn. In headless mode the browser draws video in software, which costs about one core per client and would hide the transport and codec costs that are being compared; the drawing cost is the same for both architectures for the same number of tiles, and on a real device it is done by the GPU.

5.4.2 Measurement pitfalls

Two effects would have biased the comparison if they had not been found and removed.

Loopback announcement. When the SFU announced `127.0.0.1`, the browser did not gather a loopback candidate, the UDP connectivity check failed and ICE fell back to mediasoup's passive TCP candidate. RTP over TCP suffers from head-of-line blocking, which produced frequent NACKs and high jitter without any real loss. The harness therefore announces the LAN address of the machine, and all SFU results below use UDP (the selected candidate pair is recorded with every sample). The development configuration of the application must use the LAN address for the same reason.

Probation packets. mediasoup sends padding on a dedicated stream (SSRC 1234) to probe for spare bandwidth before raising a consumer's layer. This stream shows artificial "loss" and jitter of about 200 ms, while the real media streams have zero loss and near-zero jitter; the consumer statistics on the SFU also report zero loss and zero NACK. Its bytes are counted in the downlink, because they do cross the

network, but it is excluded from the quality metrics. Without this correction the SFU would have appeared worse than the mesh.

5.4.3 Results: bandwidth

Table 5.6 and Table 5.7 give the results of the two architectures, and Figures 5.3 to 5.7 plot the main metrics against N with error bars of one standard deviation across runs.

Table 5.6. Results of the mesh baseline (mean ± SD over runs)

N	Uplink (kbps)	Downlink (kbps)	CPU (%)	Encoders	RTT (ms)	Video jitter (ms)	Loss (%)	FPS
2	526 ± 1	526 ± 1	26.0 ± 2.0	1.0 ± 0.0	0.87 ± 0.10	0.00 ± 0.00	0.00 ± 0.00	19.9 ± 0.1
4	1,582 ± 1	1,582 ± 1	81.1 ± 4.1	3.0 ± 0.0	0.88 ± 0.03	1.09 ± 0.66	0.00 ± 0.00	20.0 ± 0.1
6	1,525 ± 1,828	658 ± 599	202.1 ± 3.1	1.8 ± 0.0	3,457 ± 850.4	383.7 ± 56.99	0.23 ± 0.25	10.1 ± 3.8
8	2,279 ± 1,047	1,845 ± 807	154.0 ± 6.1	1.9 ± 0.3	5,432 ± 4,275	633.4 ± 291.5	0.01 ± 0.01	14.7 ± 0.1

Table 5.7. Results of the SFU with simulcast (mean ± SD over runs)

N	Uplink (kbps)	Downlink (kbps)	CPU (%)	Encoders	RTT (ms)	Video jitter (ms)	Loss (%)	FPS
2	1,020 ± 5	481 ± 108	25.8 ± 3.7	3.0 ± 0.0	0.19 ± 0.10	0.33 ± 0.08	0.00 ± 0.00	17.2 ± 2.3
4	999 ± 13	938 ± 12	33.8 ± 1.2	3.0 ± 0.0	0.29 ± 0.10	0.31 ± 0.20	0.00 ± 0.00	20.0 ± 0.0
6	989 ± 3	1,561 ± 9	41.7 ± 0.9	3.0 ± 0.0	0.47 ± 0.11	0.75 ± 0.19	0.00 ± 0.00	20.0 ± 0.0
8	992 ± 6	1,218 ± 19	40.9 ± 0.2	3.0 ± 0.0	0.53 ± 0.03	0.82 ± 0.07	0.00 ± 0.00	20.0 ± 0.0
10	992 ± 2	1,542 ± 2	47.1 ± 1.8	3.0 ± 0.0	1.00 ± 0.11	1.83 ± 0.41	0.00 ± 0.00	20.0 ± 0.0

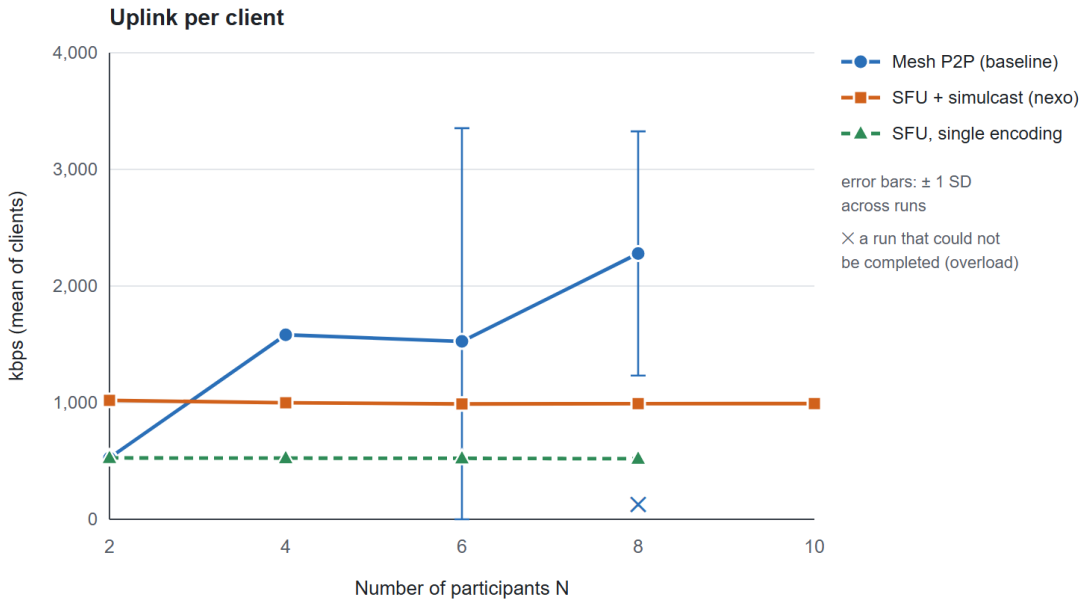


Figure 5.3. Uplink per client against the number of participants

Figure 5.3 shows the main result. The mesh uplink grows with the number of connections: each connection carries about 526 kbps, so a client uploads 526 ± 1 kbps at N = 2 and 1,582 ± 1 kbps at N = 4 with three encoders, exactly the (N−1)·b of Table 2.2. The SFU uplink is flat: 1,020 ± 5 kbps at N = 2 and 992 ± 2 kbps at N = 10, always with three encoders (the simulcast layers). At N = 2 the SFU client uploads more than the mesh client because it sends three layers instead of one; this fixed cost of simulcast is the price paid for flatness. From the mesh data, two connections at N = 3 would need about 1,053 kbps (*computed* by linear extrapolation, not measured), so the break-even point is around N = 3, and from N = 4 the SFU uploads 37% less than the mesh (*computed*).

The mesh values at N = 6 and 8 do not reflect the theoretical demand (five and seven streams, about 2,636 and 3,691 kbps, *computed*): the number of encoders that were actually sending dropped to 1.8 and 1.9 on average because connections stalled, and the variation between runs is very large. This is an overload state, not a steady state.

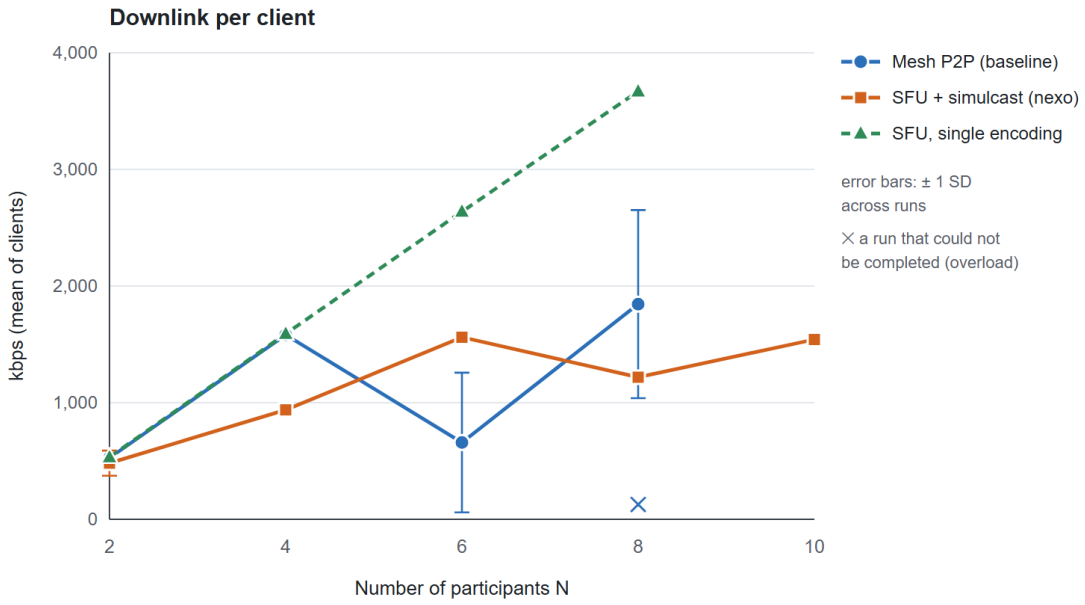


Figure 5.4. Downlink per client against the number of participants

Figure 5.4 shows the downlink. In the mesh, every received stream costs about 526 kbps and the total grows with N−1. In the SFU, the downlink follows the layer policy rather than N: one stream on layer 2 costs 481 kbps at N = 2; at N = 4 and 6 the consumers move to layer 1 and each stream costs about 312 kbps (*computed*); above six cameras they move to layer 0, so the downlink at N = 8 (1,218 kbps for seven streams) is lower than at N = 6, and at N = 10 nine streams cost 1,542 kbps. The downlink of an SFU client therefore stays bounded even as the room grows, which is the purpose of simulcast and of the layer policy.

5.4.4 Results: CPU

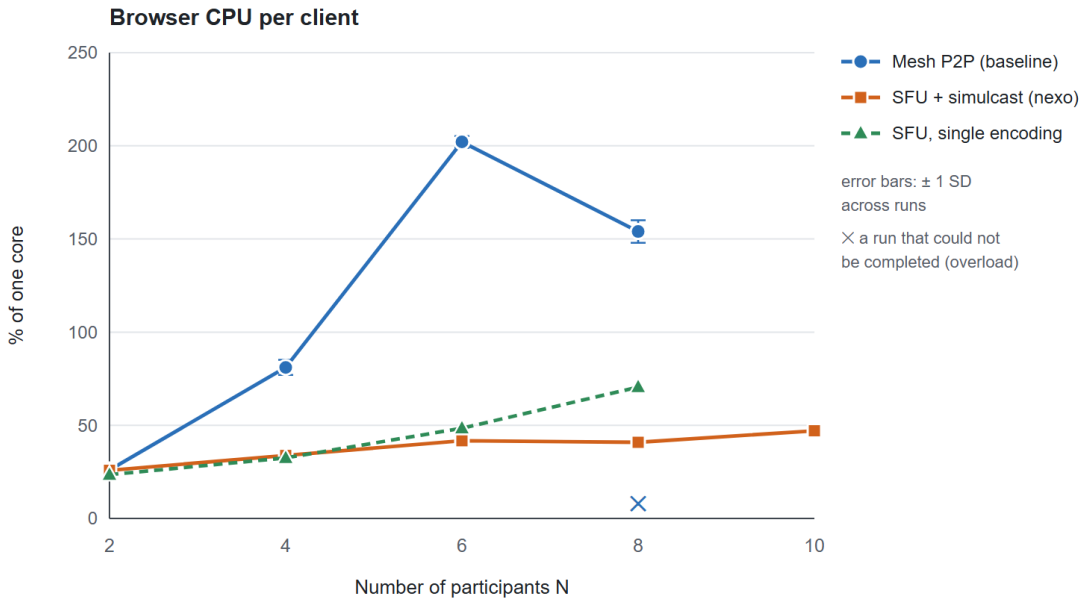


Figure 5.5. Browser CPU per client against the number of participants

Figure 5.5 compares the CPU of one browser. In the mesh, CPU grows from 26.0 ± 2.0% to 81.1 ± 4.1% of one core when N goes from 2 to 4, about 3.1 times (*computed*), in line with the number of encoders and decoders per client growing from one to three. At N = 6 a mesh browser used 202.1% on average, i.e. more than two cores, far beyond what a typical user device can spend on a call. In the SFU, CPU grows slowly from 25.8 ± 3.7% at N = 2 to 47.1 ± 1.8% at N = 10, because the number of encoders is fixed and only the decoders grow; at N = 4 the SFU client uses 58% less CPU than the mesh client (*computed*).

Table 5.8 shows the cost moved to the server. The mediasoup worker used 2.7% of one core at N = 2 and 38.9% at N = 10; it grows faster than linearly because the number of forwarded streams grows with N(N−1). The Node.js process, which carries signalling and the API, stayed between 0.1% and 1.0% of one core with 86–109 MiB of memory, which confirms the separation of control and media planes of Section 3.2.2: Node.js never touches media packets.

Table 5.8. Server-side cost of the SFU (mean over runs)

N	mediasoup worker CPU (% of one core)	Node.js CPU (% of one core)	Node.js RSS (MiB)	Downlink sum of all clients (Mbps, computed)
2	2.7 ± 0.5	0.1 ± 0.1	86 ± 0	1.0
4	10.3 ± 1.3	0.2 ± 0.1	92 ± 0	3.8
6	19.7 ± 0.9	0.2 ± 0.1	98 ± 0	9.4
8	26.6 ± 0.8	0.3 ± 0.1	102 ± 1	9.7
10	38.9 ± 2.8	1.0 ± 0.5	109 ± 0	15.4

Summing the CPU of all browsers gives the total load of the experiment (*computed* as N × mean client CPU): the mesh used about 324% of one core in total at N = 4, 1,213% at N = 6 and 1,232% at N = 8, whereas the SFU used 135% at N = 4 and 471% at N = 10, plus 38.9% for its worker. The mesh at N = 6 and 8 used most of the 16 hardware threads of the machine; part of its collapse is therefore caused by the test machine itself, a point discussed in Section 5.7.

5.4.5 Results: delay and quality

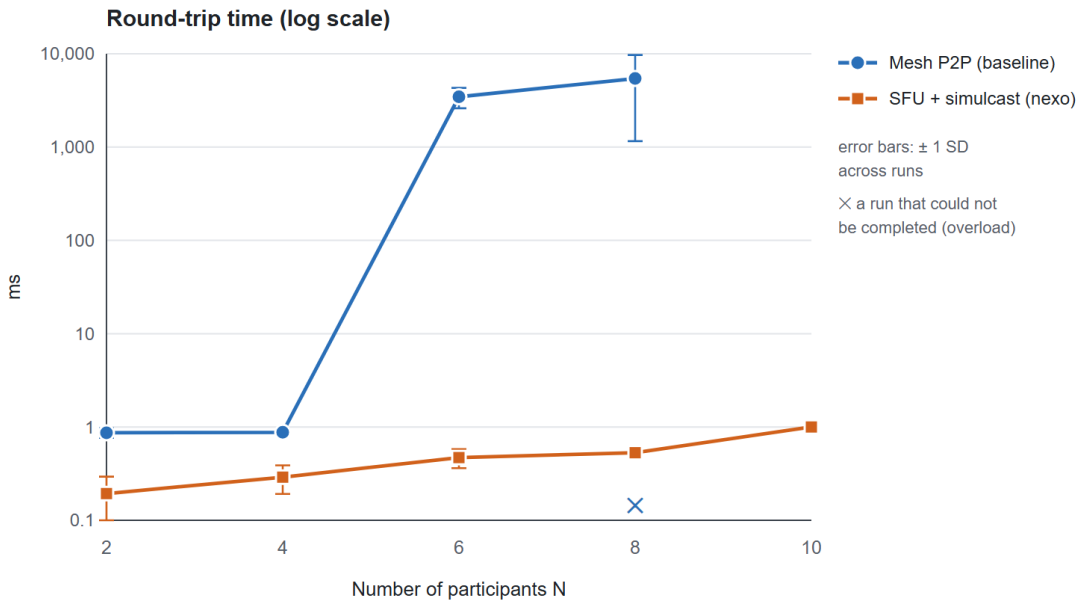


Figure 5.6. Round-trip time against the number of participants (logarithmic scale)

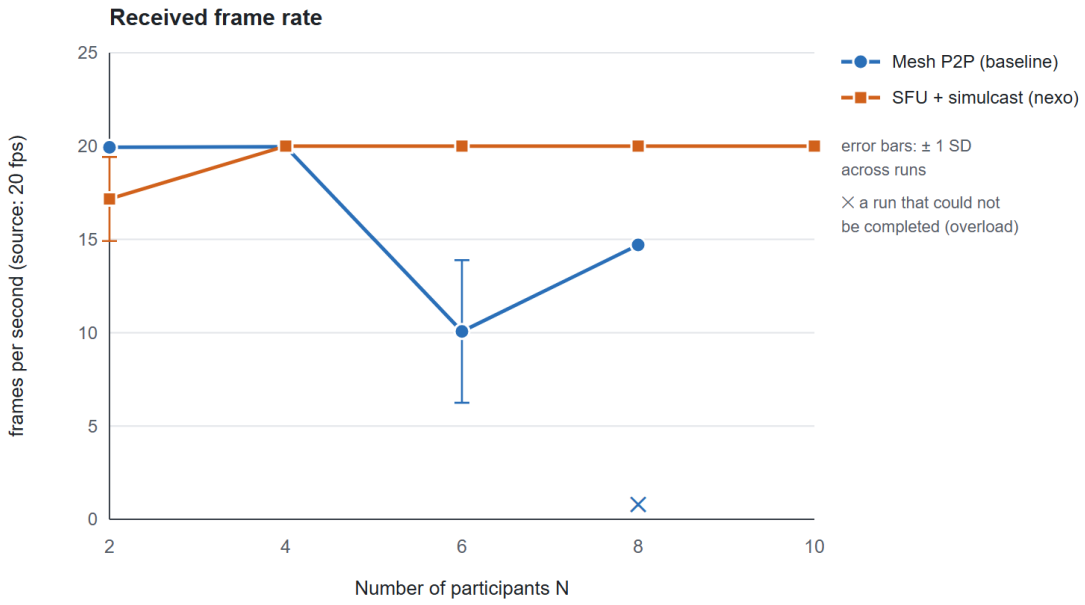


Figure 5.7. Received frame rate against the number of participants

Figure 5.6 and Figure 5.7 show the delay and the frame rate. With the SFU, RTT stayed between 0.19 and 1.00 ms, video jitter at most 1.83 ms, audio jitter at most 1.06 ms, packet loss 0.00% and NACK rate zero at every level, and from N = 4 upwards the received frame rate stayed at 20.0 fps, the rate of the source. N = 2 is the exception, with an average of 17.2 ± 2.3 fps: in every run at least one of the two receivers spent part of the window (7 to 20 of its 30 samples) below 18 fps, down to 5 fps, and the downlink was correspondingly lower and more variable. This is most likely the bandwidth estimate still rising towards the full-resolution layer, which the policy selects when there is only one remote camera and which needs the largest bitrate; it did not occur at any larger N, where lower layers are used. Holding a single-camera room on layer 1 until the estimate is sufficient would remove this effect. No video freeze occurred during the measurement windows of the SFU runs; the few freezes counted while the room was filling up correspond to layer switches when the room grew beyond six cameras.

The mesh reached similar quality at N = 2 and 4 (RTT about 1 ms, no loss, 20 fps) but **collapsed at N = 6**: RTT rose to 3,457 ± 850.4 ms, the jitter buffer held 4,077 ± 3,833 ms of media, the received frame rate fell to 10.1 ± 3.8 fps, audio jitter reached

90.92 ms, each client saw 31 video freezes per minute, and the browser reported its sending quality as limited by bandwidth or CPU during 27% of the samples. At N = 8 one of the three runs could not even be completed because the browsers stopped responding to the harness, and the completed runs show RTTs of several seconds. The first frame appeared after 5,181 ms at N = 2 and 10,648 ms at N = 8 in the mesh, against 6,228–12,719 ms in the SFU; with the SFU this time includes the sequential join of all participants through the API.

5.4.6 Effect of simulcast and layer selection

Table 5.9 and Figure 5.8 isolate the contribution of simulcast and of the layer policy. With a **single encoding**, the SFU client uploads only 526 kbps at N = 2, the same as a mesh client, but every receiver gets the full-resolution stream, so the downlink grows linearly to 3,669 kbps at N = 8, 3.0 times the downlink of the simulcast configuration at the same N (*computed*), and the client CPU rises to 70.5% because every stream is decoded at full resolution. Simulcast therefore trades a fixed uplink cost for a bounded downlink and decoding cost. Forcing **all consumers onto layer 0** lowers the downlink at N = 6 from 1,561 kbps (layer 1 under the default policy) to 857 kbps, 45% less (*computed*); at N = 10 the default policy already selects layer 0, so both configurations give about the same downlink (1,542 and 1,578 kbps). The demand-driven grid of Section 4.4.3 refines this policy per tile in the real interface, pausing tiles that are off-page and requesting larger layers only for large tiles.

Table 5.9. SFU variants: simulcast with the default policy, single encoding, and layer 0 forced (mean over runs)

Configuration	N	Uplink (kbps)	Downlink (kbps)	CPU (%)	FPS
Simulcast, default policy	2	1,020	481	25.8	17.2
Simulcast, default policy	6	989	1,561	41.7	20.0
Simulcast, default policy	8	992	1,218	40.9	20.0
Simulcast, default policy	10	992	1,542	47.1	20.0

Configuration	N	Uplink (kbps)	Downlink (kbps)	CPU (%)	FPS
Single encoding	2	526	533	23.5	20.0
Single encoding	4	524	1,590	32.5	20.0
Single encoding	6	523	2,638	48.4	20.0
Single encoding	8	519	3,669	70.5	20.0
Simulcast, layer 0 forced	6	999	857	32.0	20.0
Simulcast, layer 0 forced	10	985	1,578	48.5	20.0

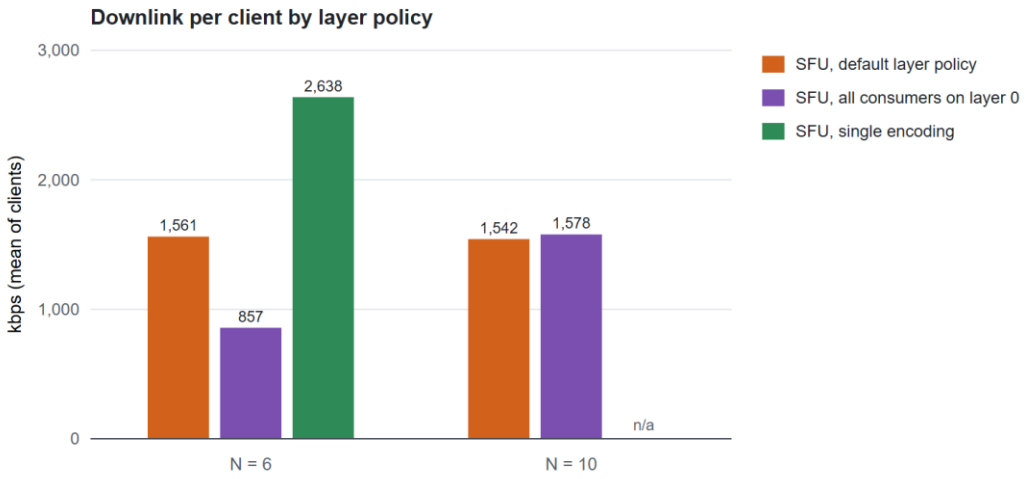


Figure 5.8. Downlink per client for the three SFU configurations at N = 6 and N = 10 (single encoding was measured up to N = 8)

5.4.7 Discussion

1. **The mesh collapses between N = 4 and N = 6, while the SFU shows no sign of degradation up to N = 10.** This matches Table 2.2: the mesh costs grow linearly with N−1 in uplink, downlink and CPU, while the SFU keeps the uplink constant and lets the downlink be controlled by the layer policy.
2. **The SFU is not cheaper at every N.** At N = 2, simulcast costs more uplink than a mesh connection. A single encoding, or peer-to-peer media for two-person calls, would be the better choice there; automatic switching is not implemented.

3. **The measured SFU uplink (about 1.0 Mbps) is lower than the 1.75 Mbps of the example in Section 2.3.4.** That figure is the sum of the maximum bitrates of the three layers. With the simple synthetic content at 960×540 and 20 fps the encoder does not use the full budget; real cameras would push the uplink closer to the ceiling. The result that matters is that the uplink does **not** depend on N.
4. **The cost moves to the server.** The SFU's outgoing bandwidth is the sum of the downlinks of all clients (about 15.4 Mbps at N = 10, *computed*), and the worker's CPU grows faster than linearly with N. An SFU server must be provisioned for this bandwidth, a cost that a mesh does not have.

5.5 Comparison with existing systems

Besides the measured mesh baseline, Table 5.10 places nexo among well-known platforms. Persistent channels with integrated voice and video are the model of Discord and Slack [2], [1]; dedicated meeting products such as Jitsi Meet [28] focus on the meeting itself. Like Jitsi Videobridge and Discord's media servers, nexo uses a selective forwarding unit, which is the architecture that open-source SFU studies [23] and the results above identify as the scalable choice for multi-party calls. Jitsi's Last N [22] forwards only the most recently active speakers; nexo instead derives the forwarded streams and their layers from the tiles that are actually visible, a complementary policy that could be combined with speaker activity in future work.

Table 5.10. Qualitative comparison with existing platforms

Feature	nexo	Discord	Slack	Jitsi Meet
Workspaces with persistent text channels	✓	✓	✓	–
Video calls inside the workspace	✓ (video channels)	✓ (voice/video channels)	✓ (huddles)	– (separate meetings)
Media architecture	SFU (mediasoup)	SFU (own servers)	Proprietary	SFU (Jitsi Videobridge)
Self-hosted, source available	✓	–	–	✓

Feature	nexo	Discord	Slack	Jitsi Meet
Waiting room and host moderation	✓	Partial	Partial	✓
Task board linked to messages	✓	–	Via integrations	–

The comparison is qualitative: the commercial platforms run on global infrastructure that cannot be reproduced on one machine, so no performance comparison with them is claimed.

5.6 Requirement verification

Table 5.11 checks the non-functional requirements of Table 3.3 against the measurements.

Table 5.11. Verification of the non-functional requirements

Requirement	Measured result	Assessment
Messages delivered in < 100 ms (p95) on a local network	Receipt p95 32.2 / 34.9 / 45.9 ms at 50 / 100 / 200 sockets; ACK p95 at most 48.6 ms; p99 at 200 sockets about 140 ms	**Met** for p95 on one host; the p99 tail at 200 sockets exceeds 100 ms
Client uplink and CPU do not grow with N (SFU)	Uplink 1,020 → 992 kbps for N = 2 → 10; CPU 25.8% → 47.1%	**Met for uplink**; CPU grows slowly with the number of decoded streams, far below the linear growth of the mesh
Loss ≈ 0, jitter < 30 ms, RTT < 150 ms on a good network	SFU: loss 0%, video jitter ≤ 1.83 ms, audio jitter ≤ 1.06 ms, RTT ≤ 1.00 ms up to N = 10	**Met** on the loopback path; not yet measured on real networks
No loss or duplication after lost ACKs or reconnection; ICE recovery	0 lost, 0 duplicated in 732 messages with 36 retries and 6 reconnections; ICE restart implemented and tested at signalling level	**Met** in the tested scenarios

Requirement	Measured result	Assessment
Hashed passwords, short-lived tokens, per-event authorisation, DTLS-SRTP	Cross-permission tests pass; DTLS-SRTP is mandatory in WebRTC	**Met** for authorisation; no packet capture yet to show DTLS-SRTP on the wire
Several API processes; meetings placed on workers	Two API processes and a killed worker in integration tests	**Met** for several processes and workers on one host; multi-node SFU not validated

5.7 Limitations of the measurements

The following limitations define what may be concluded from this chapter.

1. **One machine.** All browsers, the API, PostgreSQL and the SFU shared 16 hardware threads. When the total CPU demand approached the capacity of the machine, every metric degraded because of contention. The mesh at N = 6 and 8 was in this state, so part of its collapse comes from the test machine; on real devices each participant has its own CPU, and the breaking point of a mesh is set by the uplink or the CPU of the weakest device, which may lie at a different N. The robust conclusions are the trends (linear growth for the mesh, flat uplink for the SFU), not the exact breaking point.
2. **Loopback network.** There is no real loss, delay or bandwidth limit, so RTT, jitter and loss reflect processing load rather than network quality. Multi-network tests, TURN and controlled impairment were not performed.
3. **Synthetic media and no rendering.** The synthetic camera produces simple content and therefore lower bitrates than a real camera, and the cost of drawing video was excluded. Real CPU usage would be higher by about the same amount for both architectures.
4. **Three runs per level.** Standard deviations are based on three runs per level (two for the mesh at N = 8, where one run could not be completed), which is enough to show trends but not to give tight confidence intervals.

5. **No packet capture.** The packet-capture analyser and procedure exist in the repository, but no capture was made, so there is no packet-level evidence for STUN, DTLS and RTP/RTCP.
6. **Moderate messaging load.** The messaging test used 4 messages per second; soak tests, saturation tests, long network outages and mobile networks were not simulated.
7. **Infrastructure not validated.** Remote CI, production backups, cloud S3 and e-mail, public HTTPS and a real TURN server were outside the tested environment, so the system is not claimed to be production-ready.

5.8 Chapter summary

The automated tests confirmed the correctness of message idempotency, permissions, administration, worker fault tolerance, multi-process operation, migrations and recovery. The messaging core kept the 95th-percentile receipt latency below 46 ms with 200 sockets and lost or duplicated no message. In the main experiment, the SFU kept the client uplink at about 1 Mbps and the client CPU low from N = 2 to N = 10 with no packet loss and a stable 20 fps from N = 4 upwards, whereas the mesh baseline grew linearly and collapsed at N = 6 on the same machine. These results were obtained on one host over the loopback network; extending them to real networks is the main item of future work.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Achieved results against the objectives

Chapter 1 set four objectives. Table 6.1 restates each of them, summarises what was achieved and how it was verified, and states the level reached.

Table 6.1. Objectives of Chapter 1 and the level achieved

Objective	Result	Evidence	Level
O1 – Integrated workspace	A web application organised as Workspace → Channel with text, voice and video channels, private channels, invitations, roles, administration and audit, plus task board, notifications, search and profiles	Chapters 3–4; browser acceptance tests; Figures 4.6–4.10	Achieved
O2 – Real-time messaging and presence	Idempotent delivery (client identifier, IndexedDB outbox, advisory lock and unique constraint), backfill after reconnection, presence with TTL across processes, typing indicators	Section 5.2; Section 5.3: 0 lost and 0 duplicated messages, p95 receipt latency ≤ 46 ms at 200 sockets	Achieved on a single machine; not yet measured on a wide-area network
O3 – Multipoint video conferencing	mediasoup SFU with a worker pool, three-layer simulcast, demand-driven layers and pausing, ICE recovery, time-limited TURN credentials, waiting room and moderation	Section 5.4: SFU uplink 989–1,020 kbps from N = 2 to N = 10 with no packet loss (20 fps from N = 4); the mesh baseline collapsed at N = 6	Achieved in the laboratory; multi-network and TURN tests remain

Objective	Result	Evidence	Level
O4 – Quantitative evaluation	Media laboratory with real browsers, repeated runs and raw data; load test of the messaging core; comparison with a mesh baseline	Chapter 5, raw data in `docs/evidence/`	Achieved for a single host; packet-level capture and multi-host runs remain

The central technical claim of the thesis, that an SFU keeps the cost of each participant independent of the size of the room while a mesh does not, was confirmed experimentally on the same hardware. From N = 2 to N = 10 the uplink of an SFU client stayed between 989 and 1,020 kbps and its CPU below 47% of one core, while the CPU of a mesh client grew 3.1 times from N = 2 to N = 4 and the mesh session collapsed at N = 6.

6.2 Contributions

The main contributions of the project are:

1. A working, self-hostable collaboration platform that integrates persistent channel messaging and multipoint conferencing in one browser application, with a shared permission model enforced on every REST request and every socket event.
2. A message delivery protocol that combines client-generated identifiers, a durable outbox, a transaction-scoped advisory lock and a uniqueness constraint to obtain exactly-once storage on top of at-least-once delivery, verified under lost acknowledgements, concurrent retries and reconnections.
3. A demand-driven video grid that derives the simulcast layer of every consumer from the size of its tile and pauses consumers that are not displayed, so that the downlink follows what the user sees rather than the size of the room.

4. A reproducible media laboratory that runs every participant as a real browser, compares the SFU with a mesh baseline under identical conditions, repeats each level and keeps all raw samples, together with documented measurement pitfalls (loopback announcement and probation packets) that would otherwise bias the results.

6.3 Limitations

- All media and messaging measurements were taken on one machine over the loopback path of its LAN address. They show trends and relative costs reliably, but they do not measure behaviour under real network delay, loss or bandwidth limits.
- No packet capture was made, so DTLS-SRTP and RTCP behaviour is supported by the specifications and by `getStats()` but not by packet-level evidence.
- The synthetic camera produces simple content; real cameras would produce higher bitrates, and the rendering cost of video was excluded from the CPU measurements.
- Multi-node deployment of the SFU, TURN on a public server, production e-mail delivery, cloud storage, remote continuous integration and production backups have been implemented or configured but not validated on real infrastructure. The system is therefore not claimed to be production-ready.
- There are no native mobile applications, no recording and no end-to-end encryption through the SFU.

6.4 Future work

1. **Multi-network evaluation.** Repeat the media experiments with clients on several machines and networks, including a client that must use TURN, and with controlled impairment (loss, delay, bandwidth limits) at a router;

capture packets with Wireshark to analyse STUN, DTLS, RTP and RTCP (SR/RR, NACK, PLI) and to cross-check `getStats()`.

2. **Adaptive topology for small rooms.** Use a single encoding or peer-to-peer media for two-person calls, where simulcast costs more uplink than it saves, and switch to the SFU when a third participant joins.
3. **Active-speaker selection.** Combine the demand-driven grid with audio-level based selection of the forwarded streams (in the spirit of Last N [22]) for large rooms.
4. **Scaling the media plane.** Pipe routers between workers and nodes so that a very large meeting can span several workers, and validate the multi-node deployment with load balancing on real servers.
5. **Product features.** Direct messages, recording, end-to-end encryption with insertable streams, and native or progressive mobile applications.

In conclusion, the thesis delivered an integrated real-time workspace with in-browser multipoint conferencing and showed, with reproducible measurements, that its SFU-based media architecture scales where the peer-to-peer baseline does not. The remaining work is mainly to extend the evaluation from one machine to real networks and real infrastructure.

REFERENCES

[1] Slack Technologies, "What is a channel?," Slack Help Center. [Online]. Available: https://slack.com/help/articles/360017938993-What-is-a-channel (accessed Oct. 2026).

[2] J. Vass, "How Discord handles two and half million concurrent voice users using WebRTC," Discord Engineering Blog, Sep. 2018. [Online]. Available: https://discord.com/blog/how-discord-handles-two-and-half-million-concurrent-voice-users-using-webrtc

[3] Meta Open Source, "React documentation." [Online]. Available: https://react.dev (accessed Oct. 2026).

[4] Socket.IO, "Socket.IO v4 documentation." [Online]. Available: https://socket.io/docs/v4/ (accessed Oct. 2026).

[5] I. Baz Castillo and J. L. Millán, "mediasoup v3 documentation." [Online]. Available: https://mediasoup.org/documentation/v3/ (accessed Oct. 2026).

[6] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, Dec. 2011.

[7] C. Jennings, F. Castelli, H. Boström and J.-I. Bruaroey, "WebRTC: Real-Time Communication in Browsers," W3C Recommendation. [Online]. Available: https://www.w3.org/TR/webrtc/

[8] H. Boström, V. Singh and J.-I. Bruaroey, "Identifiers for WebRTC's Statistics API," W3C Candidate Recommendation. [Online]. Available: https://www.w3.org/TR/webrtc-stats/

[9] A. Begen, P. Kyzivat, C. Perkins and M. Handley, "SDP: Session Description Protocol," IETF RFC 8866, Jan. 2021.

[10] J. Rosenberg and H. Schulzrinne, "An Offer/Answer Model with the Session Description Protocol (SDP)," IETF RFC 3264, Jun. 2002.

[11] J. Uberti, C. Jennings and E. Rescorla, "JavaScript Session Establishment Protocol (JSEP)," IETF RFC 8829, Jan. 2021.

[12] A. Keranen, C. Holmberg and J. Rosenberg, "Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal," IETF RFC 8445, Jul. 2018.

[13] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy and P. Matthews, "Session Traversal Utilities for NAT (STUN)," IETF RFC 8489, Feb. 2020.

[14] T. Reddy, A. Johnston, P. Matthews and J. Rosenberg, "Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN)," IETF RFC 8656, Feb. 2020.

[15] J. Uberti, "A REST API for Access to TURN Services," IETF Internet-Draft draft-uberti-behave-turn-rest-00, Mar. 2013.

[16] D. McGrew and E. Rescorla, "Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP)," IETF RFC 5764, May 2010.

[17] M. Baugher, D. McGrew, M. Naslund, E. Carrara and K. Norrman, "The Secure Real-time Transport Protocol (SRTP)," IETF RFC 3711, Mar. 2004.

[18] H. Schulzrinne, S. Casner, R. Frederick and V. Jacobson, "RTP: A Transport Protocol for Real-Time Applications," IETF RFC 3550, Jul. 2003.

[19] S. Holmer, H. Lundin, G. Carlucci, L. De Cicco and S. Mascolo, "A Google Congestion Control Algorithm for Real-Time Communication," IETF Internet-Draft draft-ietf-rmcat-gcc-02, Jul. 2016.

[20] J. Bankoski, J. Koleszar, L. Quillio, J. Salonen, P. Wilkins and Y. Xu, "VP8 Data Format and Decoding Guide," IETF RFC 6386, Nov. 2011.

[21] J.-M. Valin, K. Vos and T. Terriberry, "Definition of the Opus Audio Codec," IETF RFC 6716, Sep. 2012.

[22] B. Grozev, L. Marinov, V. Singh and E. Ivov, "Last N: Relevance-based selectivity for forwarding video in multimedia conferences," in Proc. 25th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV), Portland, OR, USA, 2015.

[23] E. André, N. Le Breton, A. Lemesle, L. Roux and A. Gouaillard, "Comparative study of WebRTC open source SFUs for video conferencing," in Proc. Principles, Systems and Applications of IP Telecommunications (IPTComm), Chicago, IL, USA, 2018.

[24] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-based access control models," IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[25] C. Percival and S. Josefsson, "The scrypt Password-Based Key Derivation Function," IETF RFC 7914, Aug. 2016.

[26] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[27] PostgreSQL Global Development Group, "PostgreSQL 17 documentation: NOTIFY, LISTEN and advisory locks." [Online]. Available: https://www.postgresql.org/docs/17/ (accessed Oct. 2026).

[28] Jitsi, "Jitsi Meet handbook: architecture of Jitsi Meet and Jitsi Videobridge." [Online]. Available: https://jitsi.github.io/handbook/ (accessed Oct. 2026).